

Design and Analysis of Algorithm

Module III

Let me know, if any typos and other mistakes in the shared ppt slides

Module 3

Module No. 3	Greedy and Dynamic Programming	12 Hours
Greedy– Fractional Knapsack Problem, Job Sequencing with deadlines, Prim's algorithm- Kruskal's Algorithm-Dijkstra's Algorithm, String Matching Algo. (Naive string matching, Rabin-Karp algorithm) Dynamic Programming- Computing Warshall and Floyd algorithm, Optimal Binary Search Trees, Knapsack Problem, Traveling Salesman Problem, Matrix chain multiplication, Longest Common Subsequence (LCS)		

The greedy method in DAA is a technique for solving problems by choosing the best option at each step. It's also known as a greedy algorithm.

The greedy method is a simple and straightforward way to solve optimization problems.

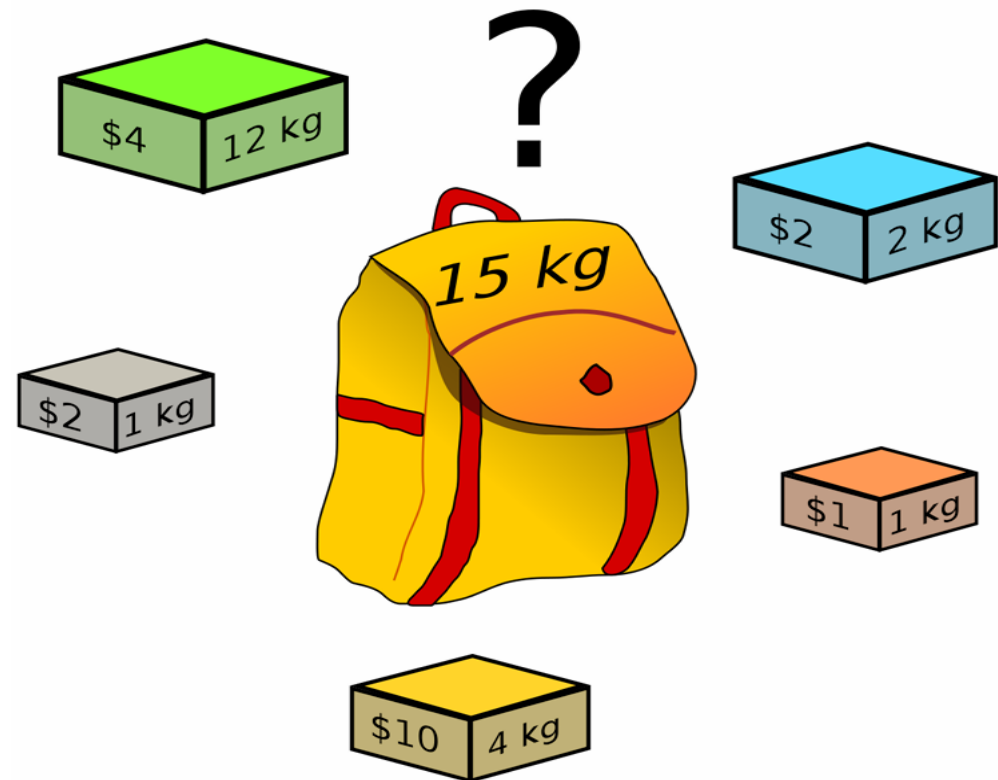
It involves making the locally optimal choice at each stage with the hope of finding the global optimum.

The main advantage of the greedy method is that it is easy to implement and understand.

On each step: the choice made must be:

- **feasible**, i.e., it has to satisfy the problem's constraints
- **locally optimal**, i.e., it has to be the best local choice among all feasible choices available on that step
- **irrevocable**, i.e., once made, it cannot be changed on subsequent steps of the algorithm

Given n items of known weights $w_1, w_2, \dots, w_n$ and values $v_1, v_2, \dots, v_n$ and a knapsack of capacity W . Find the most valuable subset of the items that fit into the knapsack.



Fractional Knapsack Problem

- Since the knapsack capacity is m , we require the total weight of all chosen objects to be at most m . Formally, the problem can be stated as

$$\begin{aligned} & \text{maximize } \sum_{1 \leq i \leq n} p_i x_i \\ & \text{subject to } \sum_{1 \leq i \leq n} w_i x_i \leq m \\ & \text{and } 0 \leq x_i \leq 1, \quad 1 \leq i \leq n \end{aligned}$$

The profits and weights are positive numbers.

- Calculate profit-weight ratio of all items(P_i/w_i)
- Select one-by-one item based on Max P_i/w_i and sum of $w_i \leq W$ pack in knapsack Bag

```
Algorithm GreedyKnapsack( $m, n$ )  
//  $p[1 : n]$  and  $w[1 : n]$  contain the profits and weights respectively  
// of the  $n$  objects ordered such that  $p[i]/w[i] \geq p[i+1]/w[i+1]$   
//  $m$  is the knapsack size and  $x[1 : n]$  is the solution vector.  
{  
    for  $i := 1$  to  $n$  do  $x[i] := 0.0$ ; // Initialize  $x$ .  
     $U := m$ ;  
    for  $i := 1$  to  $n$  do  
    {  
        if ( $w[i] > U$ ) then break;  
         $x[i] := 1.0$ ;  $U := U - w[i]$ ;  
    }  
    if ( $i \leq n$ ) then  $x[i] := U/w[i]$ ;  
}
```

Book: Sartaj Sahni

Example

- $X[] = \{x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8\}$
- $P[] = \{21, 25, 30, 28, 10, 32, 18, 20\}$
- $w[] = \{7, 8, 12, 8, 5, 10, 6, 8\}$
- $M=22$
- $P[i]/w[i] = \{21/7, 25/8, 30/12, 28/8, 10/5, 32/10, 18/6, 20/8\}$
- $P[i]/w[i] = \{3.0, 3.125, 2.5, 3.5, 2.0, 3.2, 3.0, 2.5\}$
- $P[i]/w[i] = \{3.5, 3.2, 3.125, 3.0, 3.0, 2.5, 2.5, 2.0\}$
- $X = \{x_4, x_6, x_2, x_1, x_7, x_3, x_8, x_5\}$
- Selected $[x_4 (22-8), x_6 (14-10), x_2 * (4/8)]$
- Total profit $= 28 + 32 + 25/2 \rightarrow 72.5$

Example 2

Items: $x[] = \{1, 2, 3, 4, 5, 6, 7\}$

Profit: $P[] = \{5, 10, 15, 7, 8, 9, 4\}$

Weight: $w[] = \{1, 3, 5, 4, 1, 3, 2\}$

W (Weight of the knapsack): 15

Item	Profit	Weight	P/w	Remaining weight
1	5	1	5	14-1=13
2	10	3	3.33	13-3=10
3	15	5	3	10-5=5
4	7	4	1.8	
5	8	1	8	15-1=14
6	9	3	3	5-3=2
7	4	2	2	2-2=0

Example 2



Items	1	2	3	4	5
Weights (in kg)	3	3	2	5	1
Profits	10	15	10	12	8

Knapsack capacity of 10 kg

Fractional Knapsack

```
class FractionalKnapsack {  
    public static void main (String  
args[])  
    {  
        int i,ch=0,W=10,currW,n;  
        float profit=0,max;  
        Scanner sc = new Scanner(System.in);  
        System.out.println("Enter no of items");  
        n=sc.nextInt();  
        int array[][]=new int[n][2];  
        System.out.println("Enter weight & profit  
of each items");  
        for (i=0;i<n;i++)  
        {  
            array[i][0]=sc.nextInt();  
            array[i][1]=sc.nextInt();  
        }  
        System.out.println("Enter max capacity of  
knapsack :");  
        W=sc.nextInt();
```



```
currW=W;
while (currW>0)
{
    max=0;
    for (i=0; i<n; i++)
        {if ( array[i][1] && ((float) array[i][1]) /
          ((float) array[i][0]) > max )
        {
max= ((float) array[i][1]) / ((float) array[i][0])
;
            ch=i;
        }
    }
    if (array[ch][0] < currW)
    {
        currW=currW - array[ch][0];
        profit=profit+ (float) array[ch][1];
        array[ch][13]=0;
    }
}
```

else

```
{  profit=profit+currW*max;
    currW=0;
    break;
}
}
System.out.println("The total profit is " +
profit;
}
}
```

- Complexity Analysis
- **Time Complexity:** $O(N \cdot \log N)$, where N is the number of items given.

- Job sequencing with deadlines is a problem that involves scheduling a set of jobs within their respective deadlines to maximize profit.
- This approach assumes that each job can be completed in exactly one unit of time.
- It's a Maximization Problem.

PROBLEM

- We are given a set of n jobs.
- An integer deadline $d_i \geq 0$ and a profit $p_i > 0$ is associated with each job J_i .
- For any job J_i the profit p_i is earned if the job is completed by its deadline.
- To complete a job, one has to process the job on a machine for one unit of time.
- Only one machine is available for processing jobs.

- As a first attempt we can choose the objective function $\sum_{i \in J} p_i$ as our optimization measure.
- Using this measure, the next job to include is the one that increases $\sum_{i \in J} p_i$ the most, subject to the constraint that the resulting J is a feasible solution.
- This requires to consider jobs in decreasing order of the p_i 's.

Example 1

- Jobs $j[] = \{j1, j2, j3, j4, j5\}$
- Profit $p[] = \{20, 15, 10, 5, 1\}$
- Deadline $d[] = \{2, 2, 1, 3, 3\}$
- Arrange jobs in decreasing order of P_i
- Select job with max P_i and try to assign from its deadline to 1

Job	Profit	Dead line
j1	20	2
j2	15	1
j3	10	1, already j2 is assigned
j4	5	3
j5	1	3, already j4 is assigned

Example 2

- Jobs $j[] =$
- Profit $p[] =$
- Deadline $d[] =$
- Arrange jobs in decreasing order of P_i
- Select job with max P_i and try to assign from its deadline to 1

j1	j2	j3	j4	j5	j6	j7	j8
20	18	15	10	12	22	8	25
2	3	4	1	2	2	5	3

Job	Profit	Dead line
j8	25	3
j6	22	2
j1	20	2 not possible, but can assign to 1
j2	18	3 not possible
j3	15	4
j4	10	1 not possible

20

Greedy Approach

```

1  Algorithm JS( $d, j, n$ )
2  //  $d[i] \geq 1, 1 \leq i \leq n$  are the deadlines,  $n \geq 1$ . The jobs
3  // are ordered such that  $p[1] \geq p[2] \geq \dots \geq p[n]$ .  $J[i]$ 
4  // is the  $i$ th job in the optimal solution,  $1 \leq i \leq k$ .
5  // Also, at termination  $d[J[i]] \leq d[J[i+1]], 1 \leq i < k$ .
6  {
7       $d[0] := J[0] := 0$ ; // Initialize.
8       $J[1] := 1$ ; // Include job 1.
9       $k := 1$ ;
10     for  $i := 2$  to  $n$  do  $n - 1$  times
11     {
12         // Consider jobs in nonincreasing order of  $p[i]$ . Find
13         // position for  $i$  and check feasibility of insertion.
14          $r := k$ ;
15         while  $((d[J[r]] > d[i])$  and  $(d[J[r]] \neq r))$  do  $r := r - 1$ ;
16         if  $((d[J[r]] \leq d[i])$  and  $(d[i] > r))$  then
17         {
18             // Insert  $i$  into  $J[ ]$ .
19             for  $q := k$  to  $(r + 1)$  step  $-1$  do  $J[q + 1] := J[q]$ ;
20              $J[r + 1] := i$ ;  $k := k + 1$ ;
21         }
22     }
23     return  $k$ ;
24 }
```

$J[]$ is scheduled
job list, index
indicates as slots

$\Theta(sn)$ If s jobs are
in solution

$\Theta(1)$
 k times

$\Theta(k)$

$\Theta(n^2)$ If all n jobs are in
solution- **Worst Case**

```

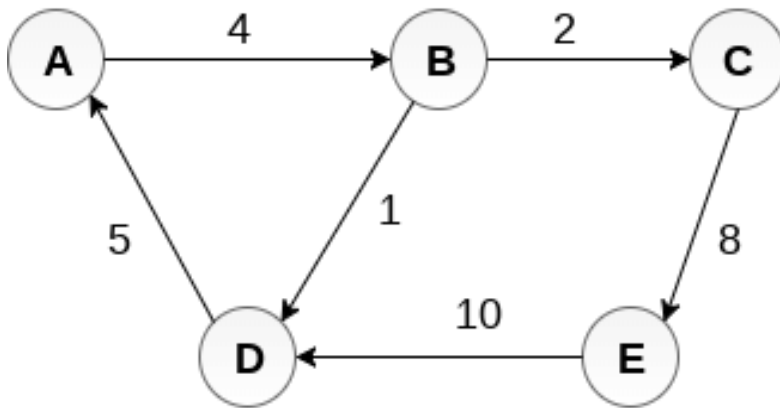
X[]=0;
max=MaximumDeadline
for(i=1; i<n; i++)
{
    k= arr[i][2];
    while (k>0)
    {
        if (x[k]= =0)
            { x[k]=arr[i][0]; break; }
        k--;
    }
}

```

	Job	P	D
0	0	0	0
1	4	20	3
2	1	18	2
3	2	18	1
4	3	15	2

	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0	4	0
2	0	0	1	4	0
3	0	2	1	4	0

MST using Greedy Method



Weighted Directed Graph

	A	B	C	D	E
A	0	4	0	0	0
B	0	0	2	1	0
C	0	0	0	0	8
D	5	0	0	0	0
E	0	0	0	10	0

Adjacency Matrix

- A spanning tree is a subset of Graph G , which has all the vertices covered with minimum possible number of edges. Hence, a spanning tree does not have cycles and it cannot be disconnected.
- A spanning tree is always defined for a graph and it is always a subset of that graph

- Given an undirected, weighted graph G , we are interested in finding a tree T that contains all the vertices in G and minimizes the sum

$$w(T) = \sum_{(u,v) \text{ in } T} w(u,v)$$

- Two most important minimum spanning tree (MST) algorithms are:
 - Kruskal's Algorithm
 - Prim's Algorithm

- Both algorithms follow the generic algorithm

GENERIC-MST(G, w)

```
1   $A \leftarrow \emptyset$ 
2  while  $A$  does not form a spanning tree
3      do find an edge  $(u, v)$  that is safe for  $A$ 
4           $A \leftarrow A \cup \{(u, v)\}$ 
5  return  $A$ 
```

Kruskal's MST Using Greedy Method

- Algorithm constructs a minimum spanning tree as an expanding sequence of subgraphs that are always **acyclic but are not necessarily connected** on the intermediate stages of the algorithm.
- The algorithm begins by sorting the graph's edges in nondecreasing order of their weights.
- Then, **starting with the empty subgraph**, it scans this sorted list, **adding the next edge on the list to the current subgraph** if such an **inclusion does not create a cycle** and simply skipping the edge otherwise

- Step 1: Sort all edges in increasing order of their edge weights.
- Step 2: Pick the smallest edge.
- Step 3: Check if the new edge creates a cycle or loop in a spanning tree.
- Step 4: If it doesn't form the cycle, then include that edge in MST. Otherwise, discard it.
- Step 5: Repeat from step 2 until it includes $|V| - 1$ edges in MST.

Kruskal's MST Algorithm

- **Step-01:**
- Sort all the edges from low weight to high weight.
- **Step-02:**
- Take the edge with the lowest weight and use it to connect the vertices of graph.
- If adding an edge creates a cycle, then reject that edge and go for the next least weight edge.
- **Step-03:**
- Keep adding edges until all the vertices are connected and a Minimum Spanning Tree (MST) is obtained.

Algorithm: Kruskal(G)

//Input: A weighted connected graph $G = \langle V, E \rangle$

//Output: E_T , the set of edges composing a minimum spanning tree of G

sort E in nondecreasing order of the edge weights $w(e_{i_1}) \leq \dots \leq w(e_{i_{|E|}})$

$E_T \leftarrow \emptyset$; $ecounter \leftarrow 0$ //initialize the set of tree edges and its size

$k \leftarrow 0$ //initialize the number of processed edges

while $ecounter < |V| - 1$ **do**

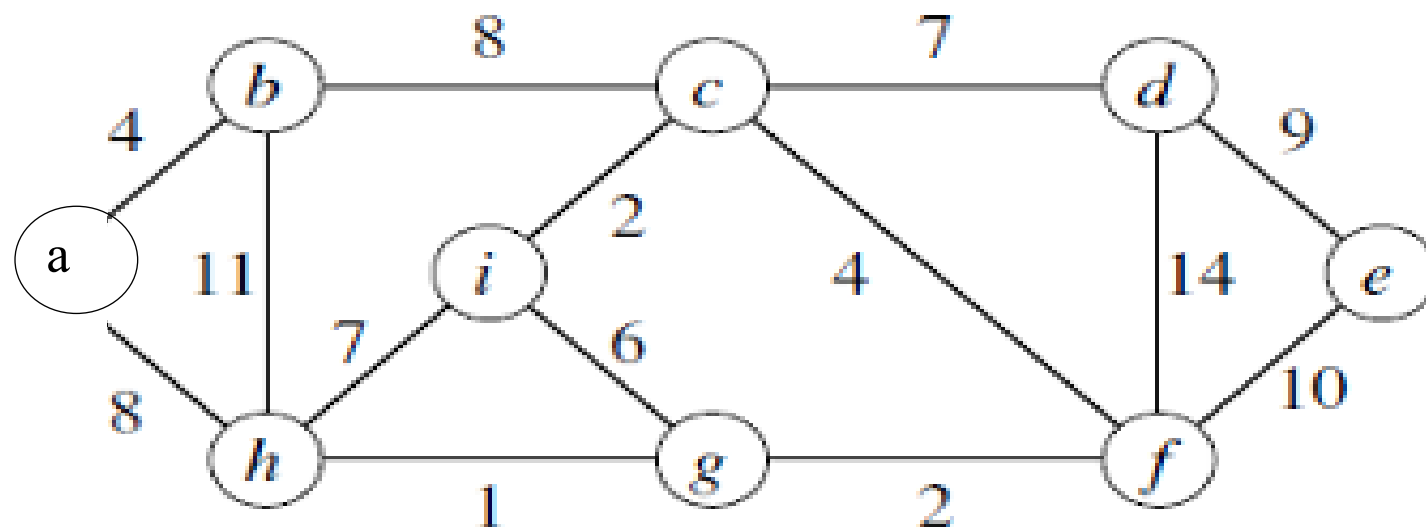
$k \leftarrow k + 1$

if $E_T \cup \{e_{i_k}\}$ is acyclic

$E_T \leftarrow E_T \cup \{e_{i_k}\}$; $ecounter \leftarrow ecounter + 1$

return E_T

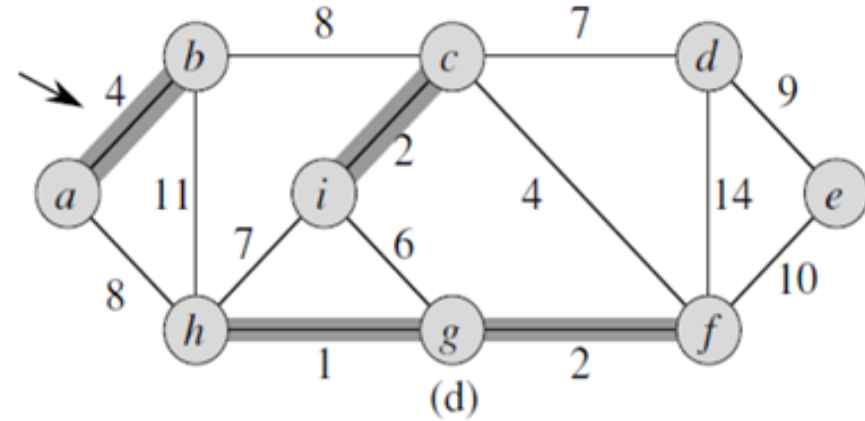
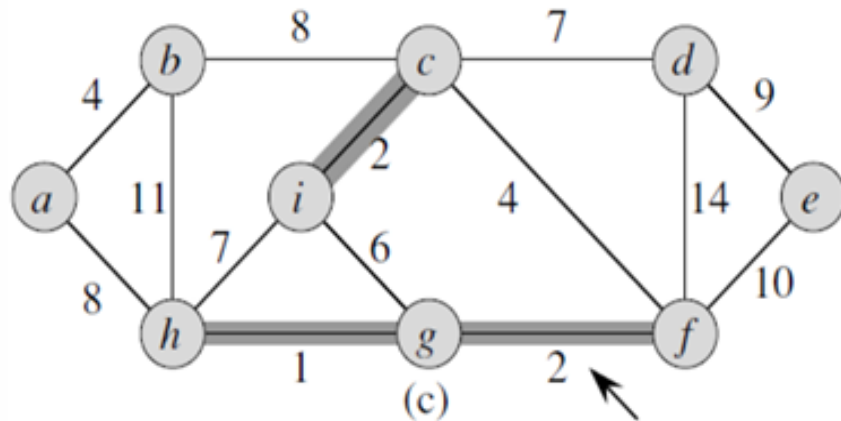
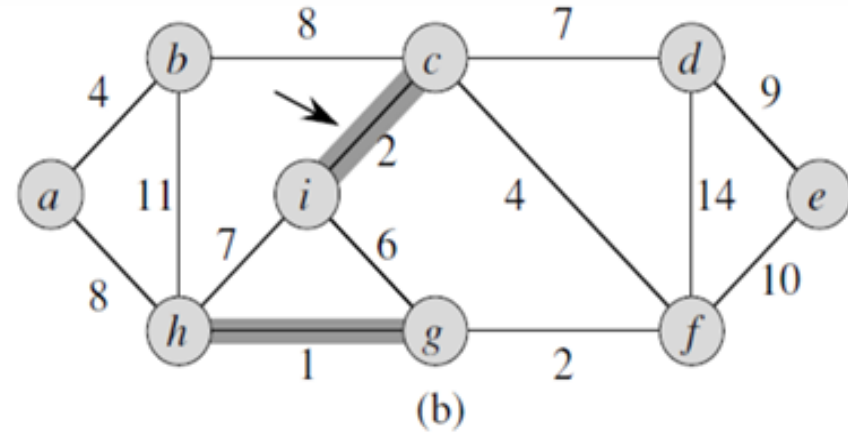
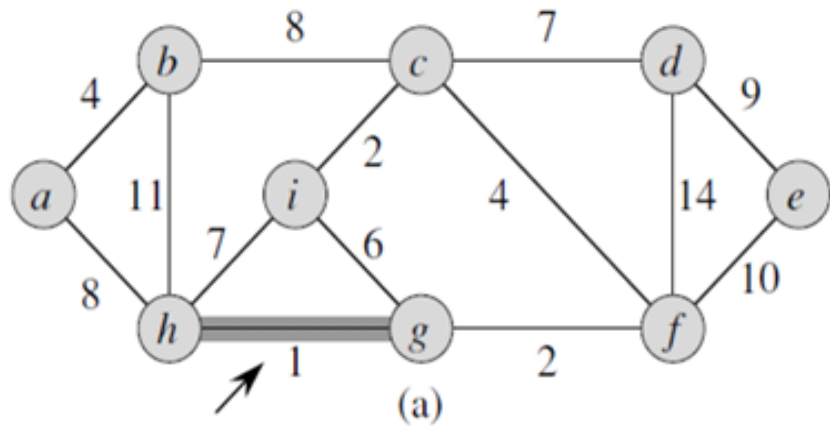
Kruskal's MST :Example

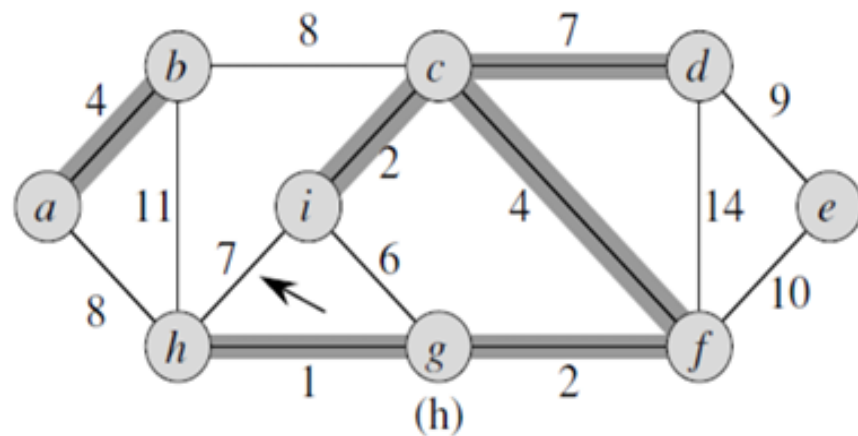
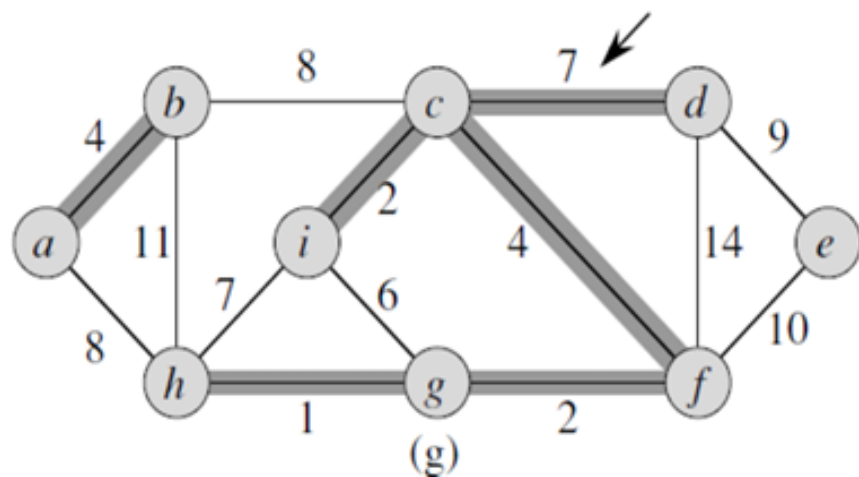
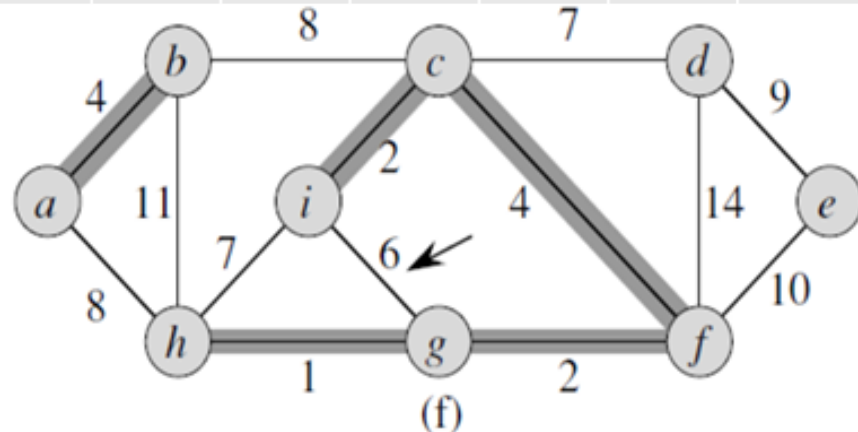
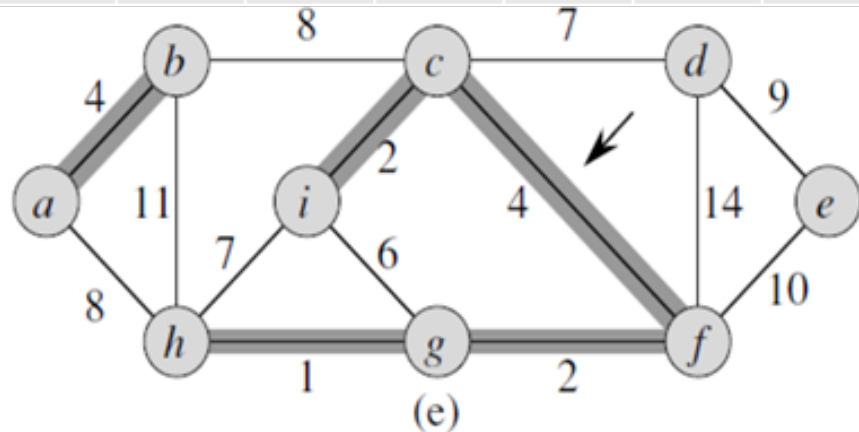


Edg e	(g,h)	(g, f)	(c, i)	(a b)	(c, f)	(i, g)	(c d)	(a,h)	(b c)	(d e)	(e,f)	(b h)	(d,f)
Wi	1	2	2	4	4	6	7	8	8	9	10	11	14

Example

Edge	(g,h)	(c,i)	(g,f)	(a,b)	(c,f)	(i,g)	(c,d)	(a,h)	(b,c)	(d,e)	(e,f)	(b,h)	(d,f)
Weight	1	2	2	4	4	6	7	8	8	9	10	11	14

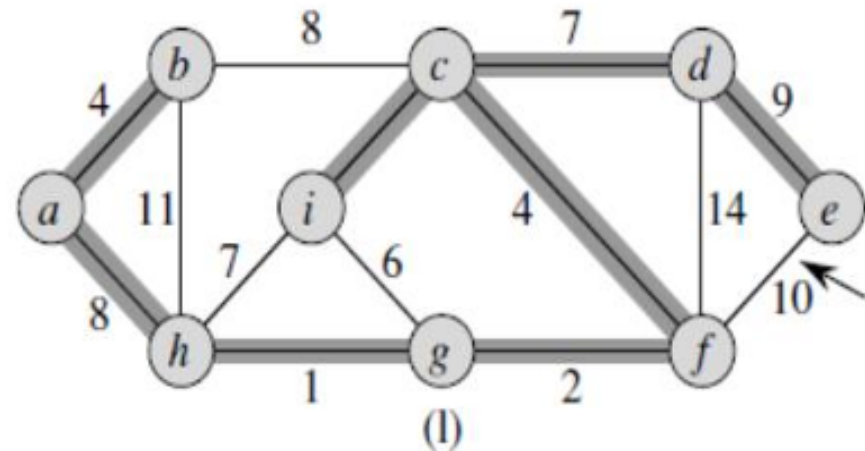
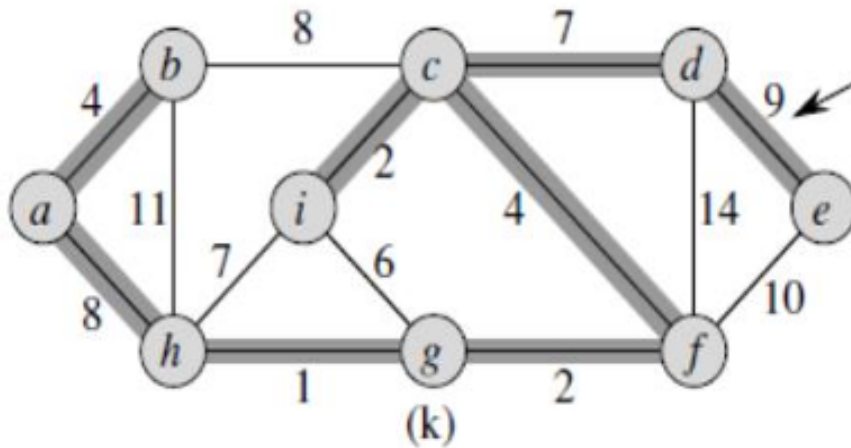
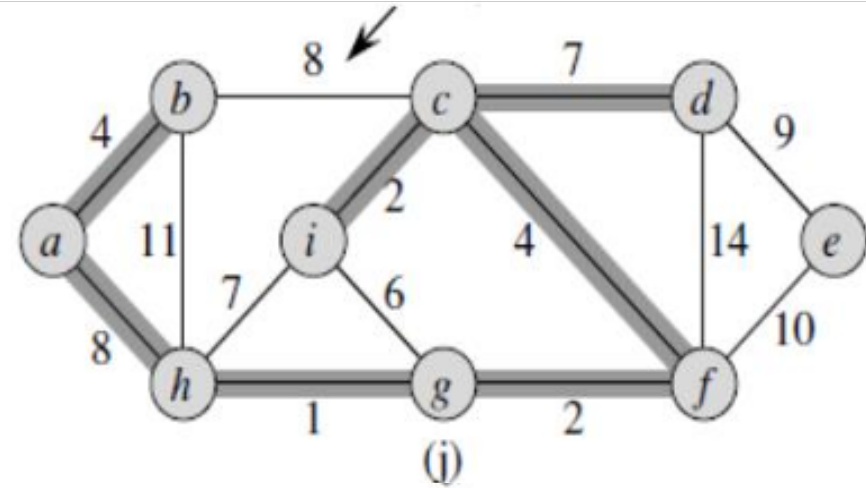
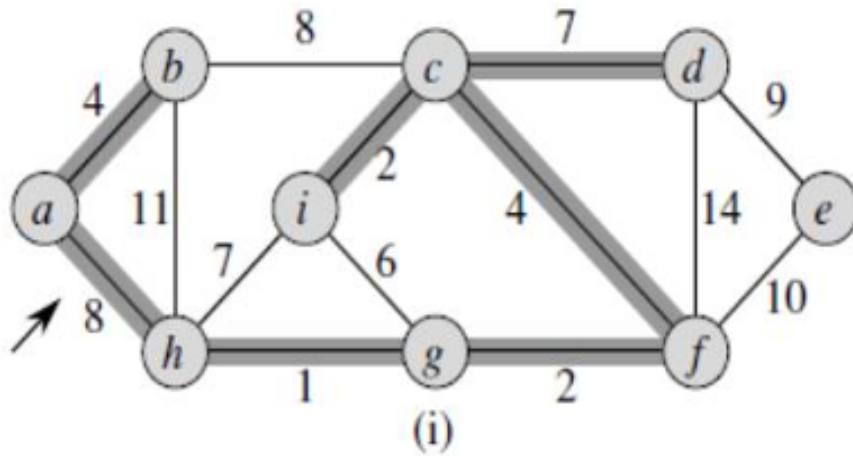




Example

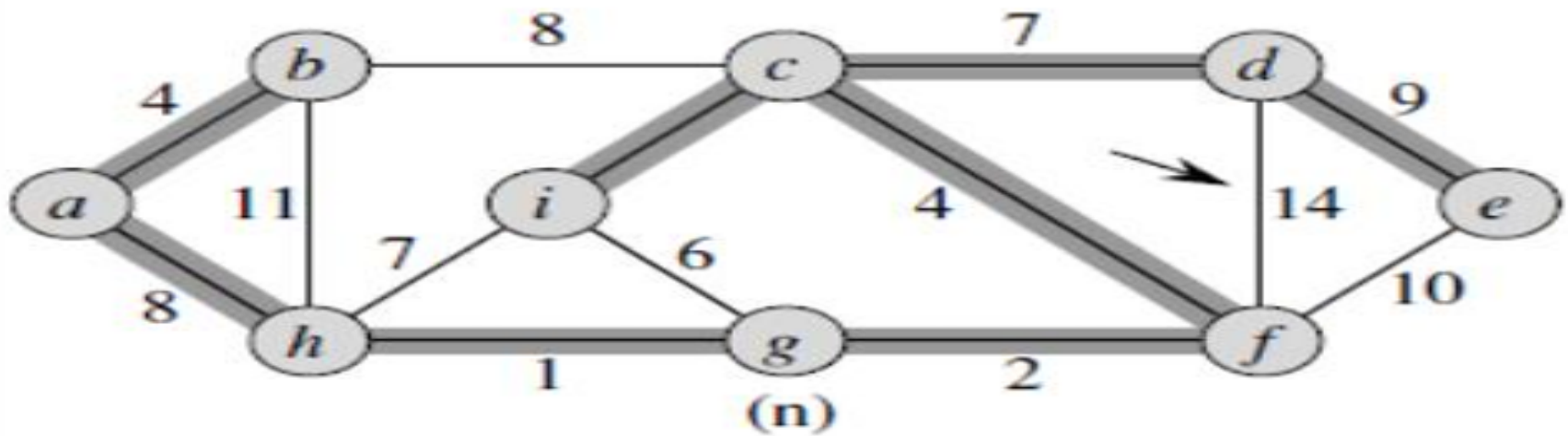
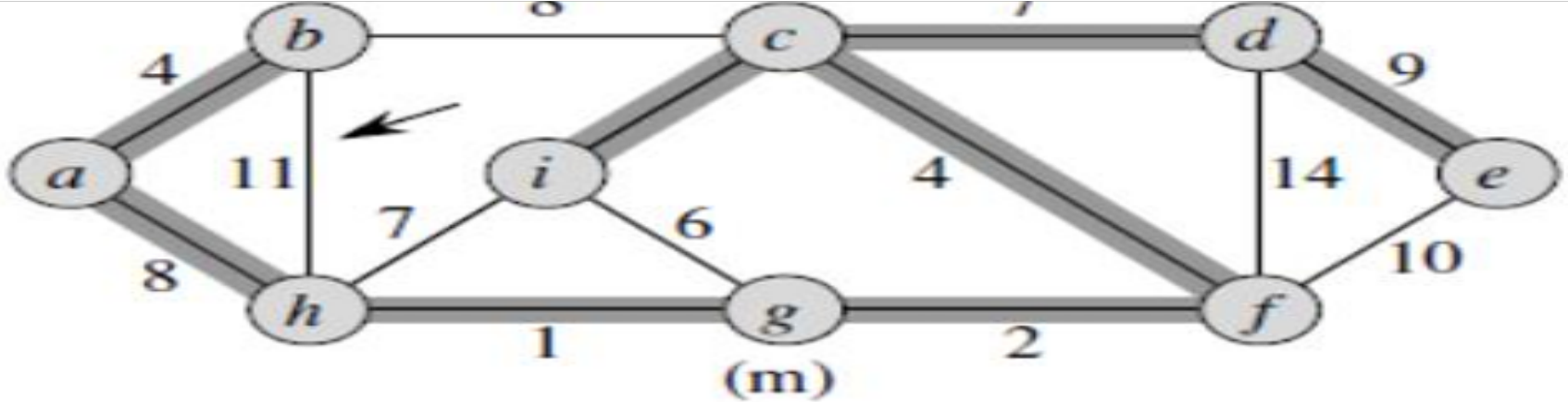


Edge	(g,h)	(c,i)	(g,f)	(a,b)	(c,f)	(i,g)	(c,d)	(a,h)	(b,c)	(d,e)	(e,f)	(b,h)	(d,f)
Wi	1	2	2	4	4	6	7	8	8	9	10	11	14

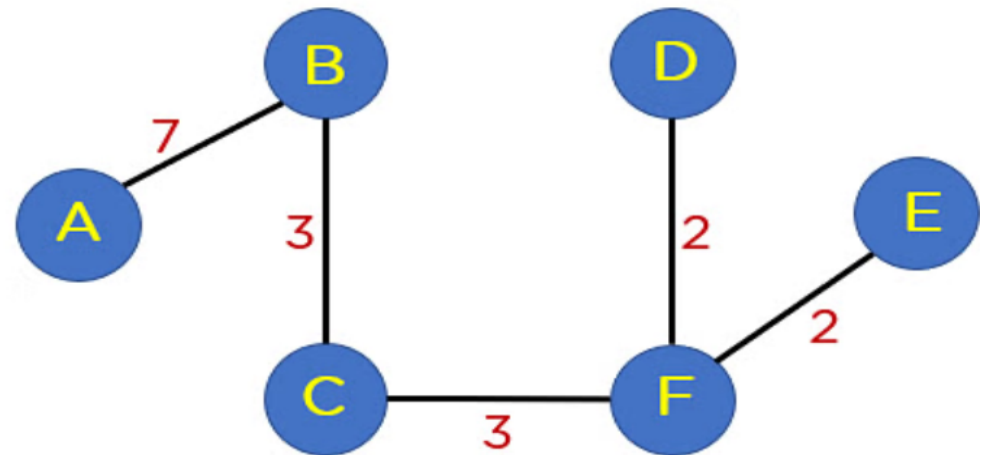
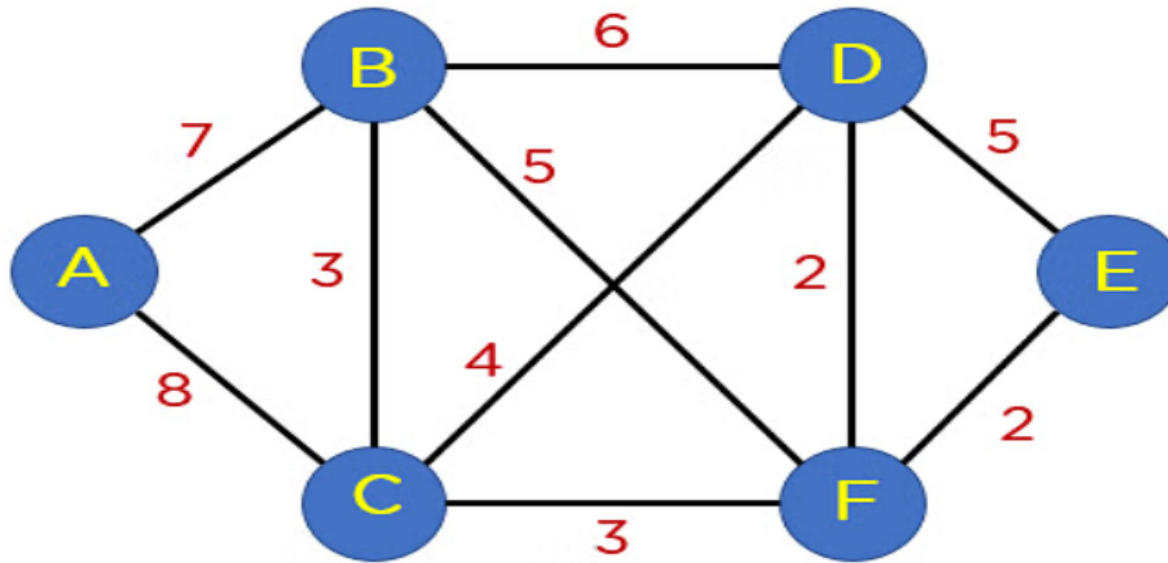


Example

Edge	(g,h)	(c,i)	(g,f)	(a,b)	(c,f)	(i,g)	(c,d)	(a,h)	(b,c)	(d,e)	(e,f)	(b,h)	(d,f)
Weight	1	2	2	4	4	6	7	8	8	9	10	11	14



Example



- **$O(E \log E)$ or $O(E \log V)$** — How?
- Sorting of edges takes **$O(E \log E)$** time.
- After sorting, we iterate through all edges and apply find-union algorithm.
- The find and union operations can take at most **$O(\log V)$** time.
- For E number of edges can take at most **$O(E \log V)$**
- So overall complexity is **$O(E \log E + E \log V)$** time.
- Therefore, overall time complexity is **$O(E \log E)$ or $O(E \log V)$**

Greedy Method

Prim's MST

- It starts with an empty spanning tree.
- The idea is to **maintain two sets of vertices**.
- The first set contains the vertices already included in the MST, the other set contains the vertices not yet included.
- **At every step**, it considers all the edges that connect the two sets, and picks the minimum weight edge from these edges.
- After picking the edge, it moves the other endpoint of the edge to the set containing MST

- Prim's algorithm constructs a minimum spanning tree through a sequence of expanding subtrees.
- The initial subtree in such a sequence consists of a single vertex selected arbitrarily from the set V of the graph's vertices.
- On each iteration, the algorithm expands the current tree in the greedy manner by simply attaching to it the nearest vertex not in that tree

Step-01: Randomly choose any vertex.

- The vertex connecting to the edge having least weight is usually selected.

Step-02:

- Find all the edges that connect the tree to new vertices.
- Find the least weight edge among those edges and include it in the existing tree.
- If including that edge creates a cycle, then reject that edge and look for the next least weight edge.

Step-03:

- Keep repeating step-02 until all the vertices are included and Minimum Spanning Tree (MST) is obtained.

ALGORITHM *Prim(G)*

//Prim's algorithm for constructing a minimum spanning tree

//Input: A weighted connected graph $G = \langle V, E \rangle$

//Output: E_T , the set of edges composing a minimum spanning tree of G

$V_T \leftarrow \{v_0\}$ //the set of tree vertices can be initialized with any vertex

$E_T \leftarrow \emptyset$

for $i \leftarrow 1$ **to** $|V| - 1$ **do**

 find a minimum-weight edge $e^* = (v^*, u^*)$ among all the edges (v, u)

 such that v is in V_T and u is in $V - V_T$

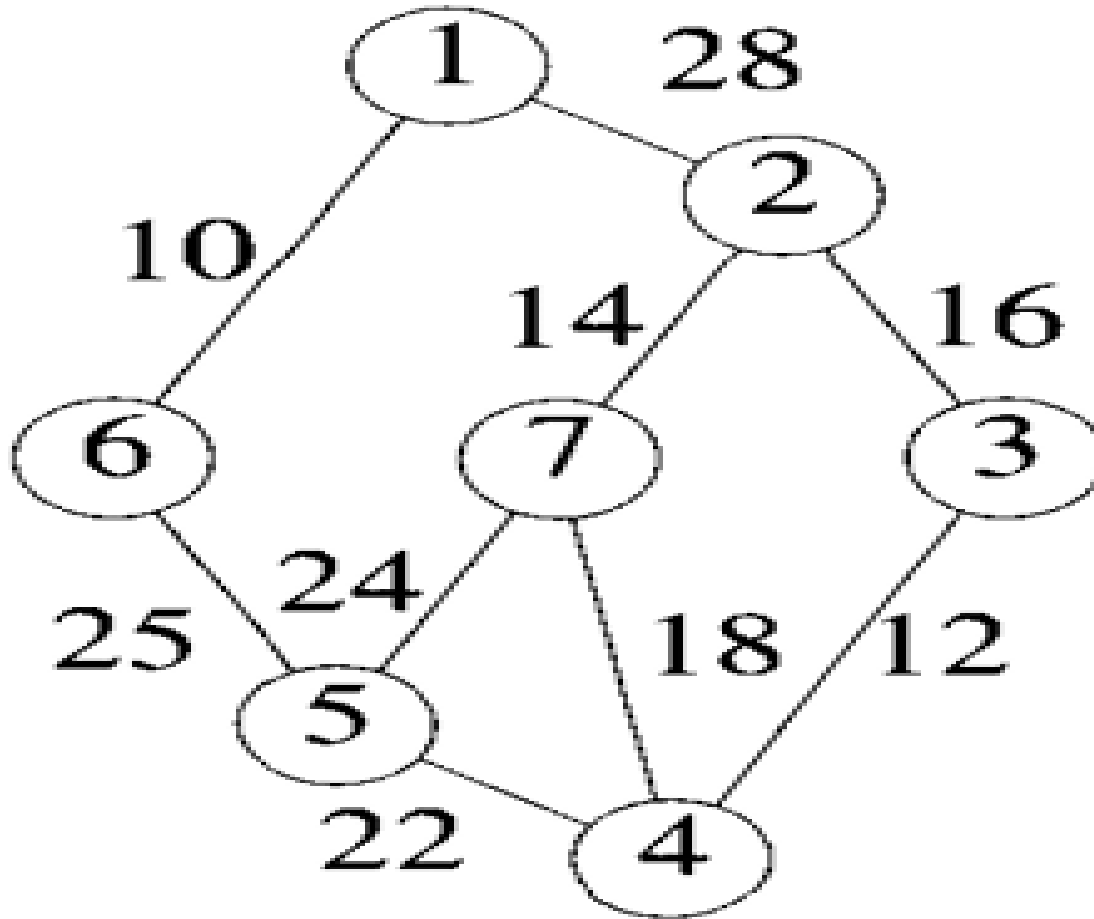
$V_T \leftarrow V_T \cup \{u^*\}$

$E_T \leftarrow E_T \cup \{e^*\}$

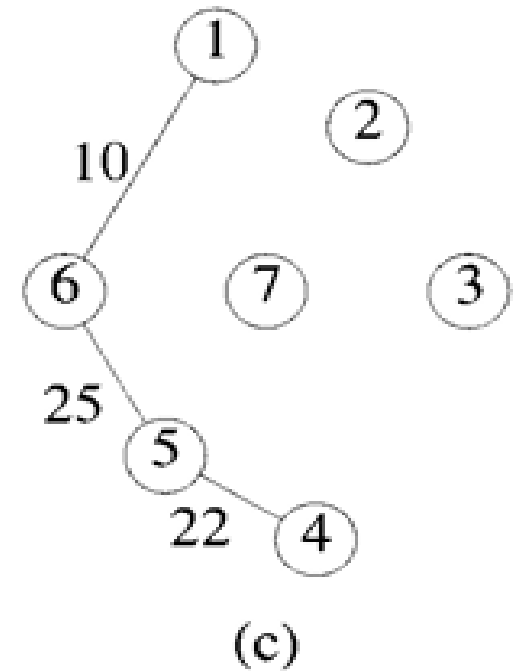
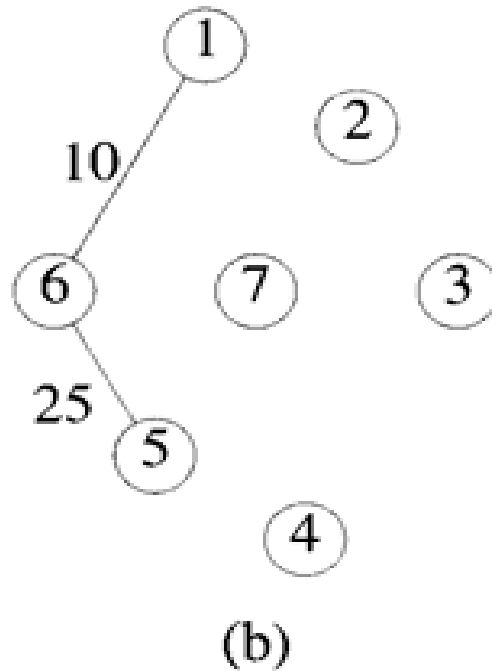
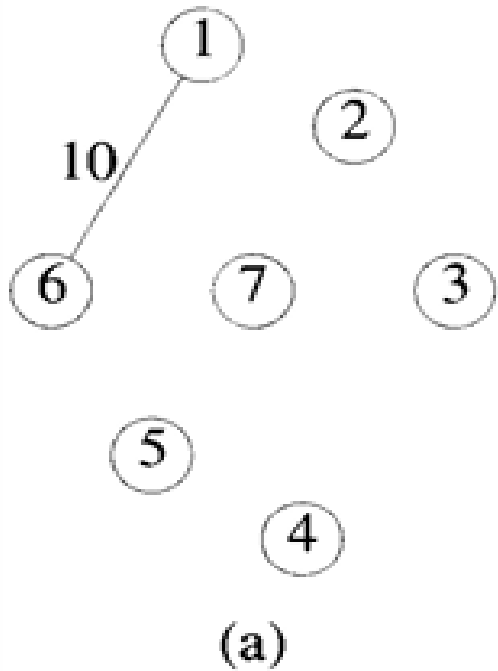
return E_T

Example 1

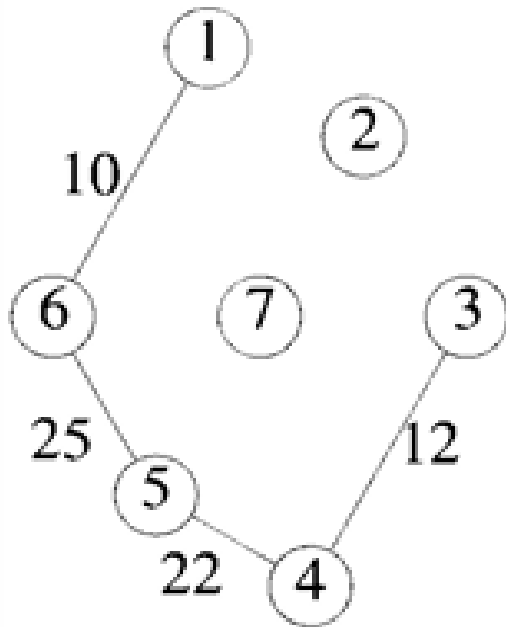
Edge	(1,6)	(6,5)	(1,2)	(5,7)	(2,7)	(2,3)	(4,5)	(4,7)	(3,4)
Wi	10	25	28	24	14	16	22	14	12



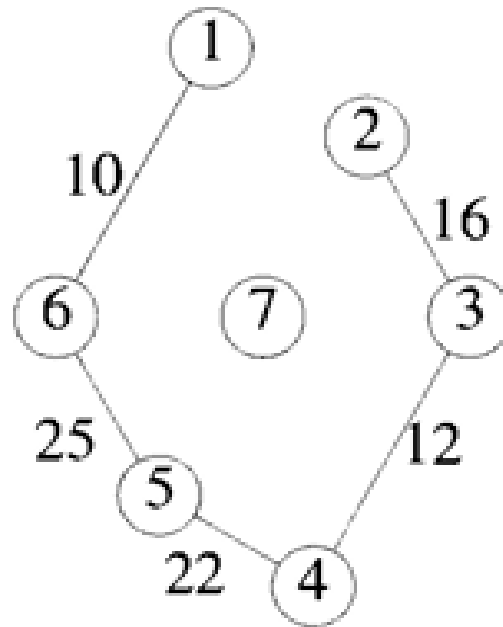
Edge	(1,6)	(6, 5)	(1,2)	(5,7)	(2,7)	(2,3)	(4,5)	(4,7)	(3,4)
Wi	10	25	28	24	14	16	22	14	12



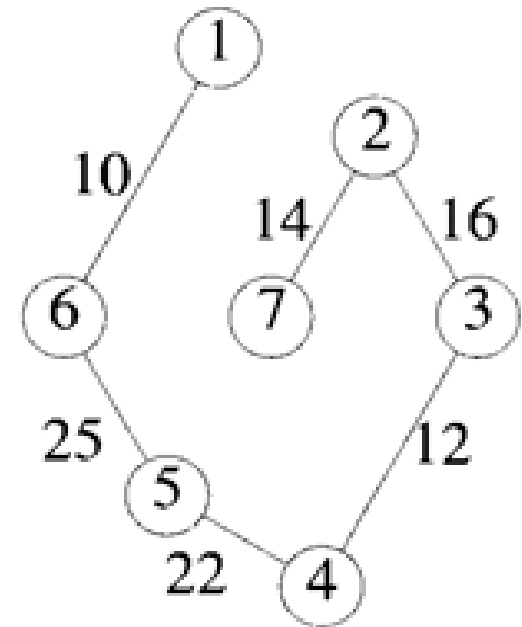
Edge	(1,6)	(6, 5)	(1,2)	(5,7)	(2,7)	(2,3)	(4,5)	(4,7)	(3,4)
Wi	10	25	28	24	14	16	22	14	12



(d)

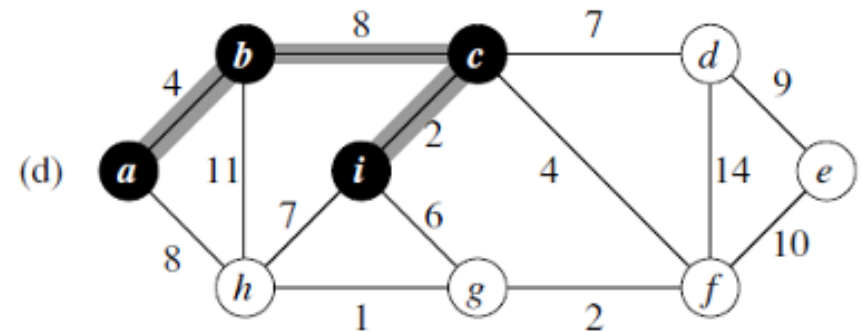
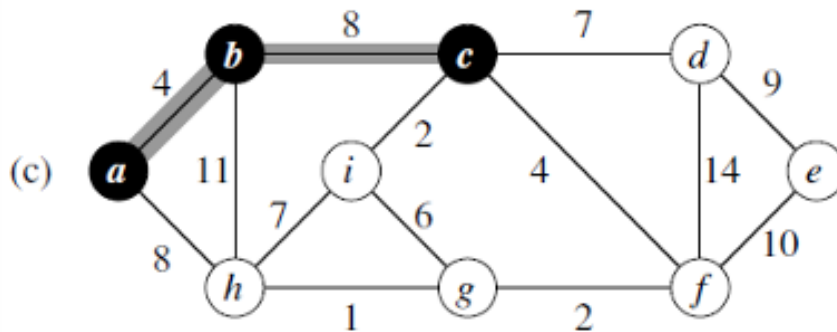
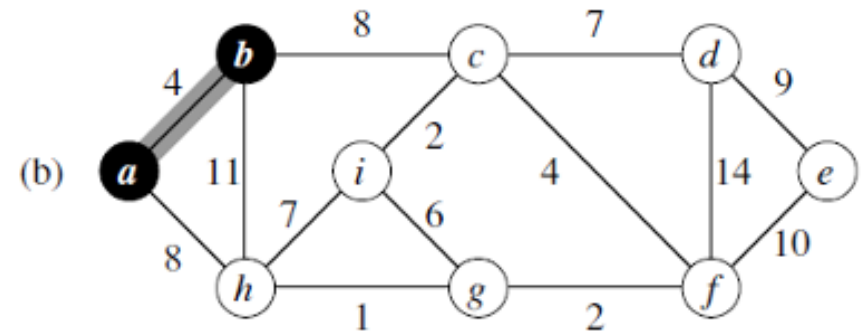
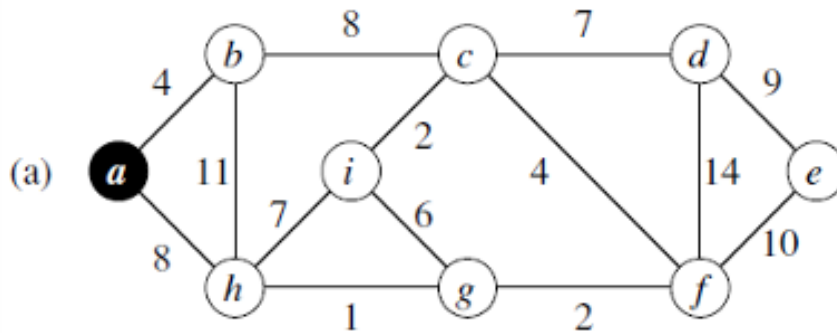
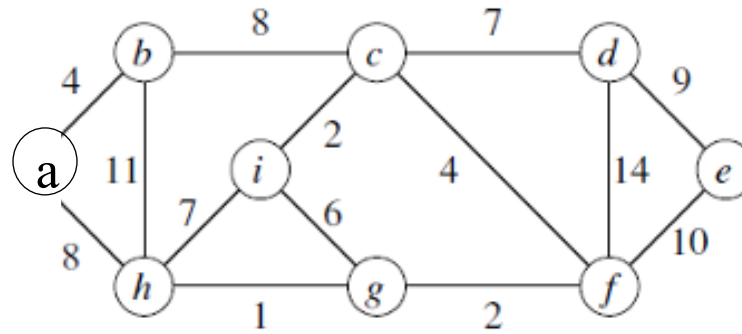


(e)

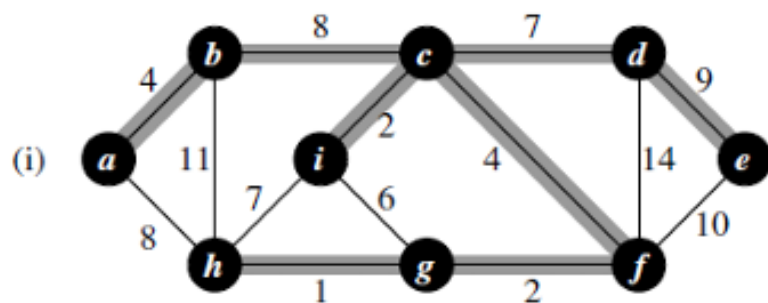
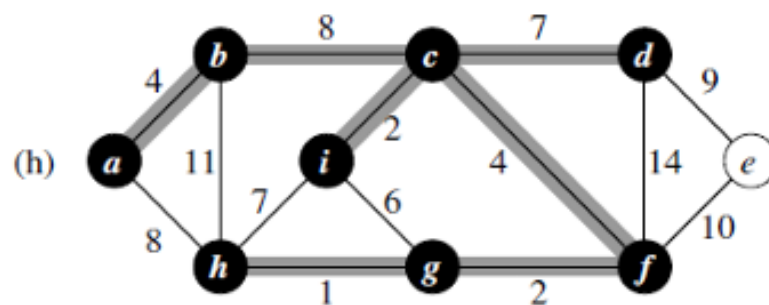
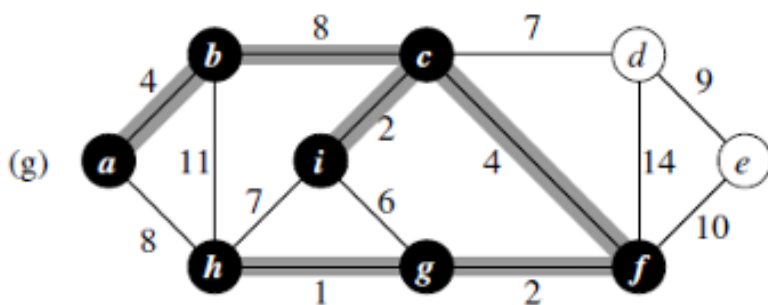
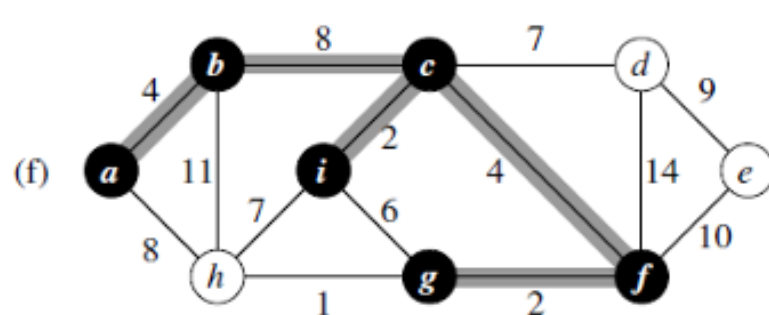
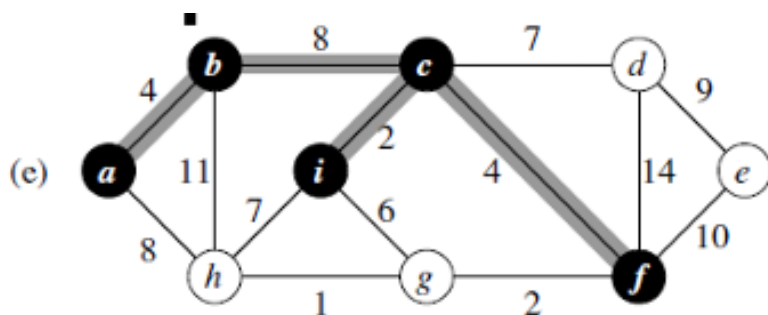


(f)

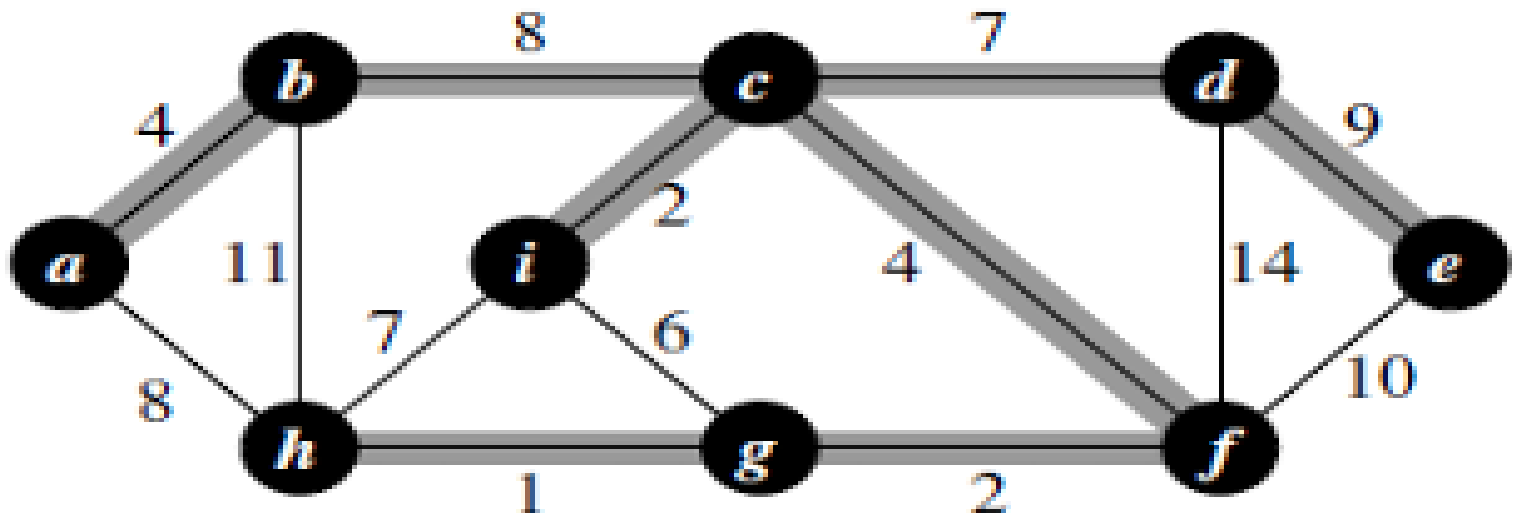
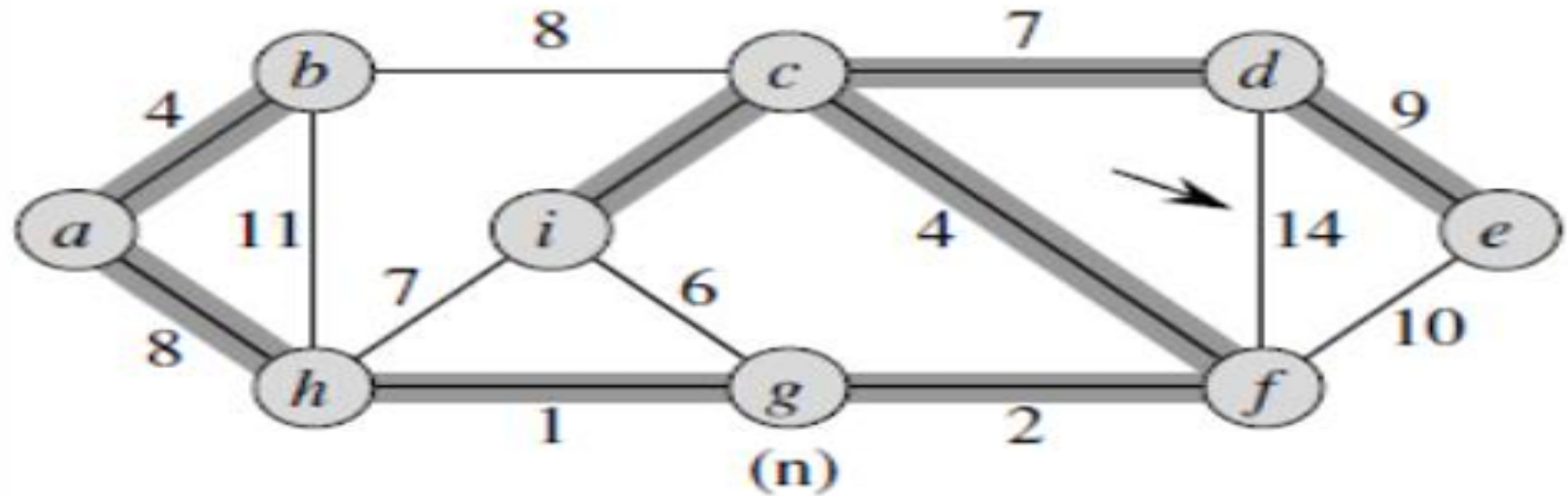
Example 2



Example 2



Kruskal Vs prim



- Time Complexity of the above program is $O(V^2)$, if an adjacency matrix is used as input.
- This is because, in each iteration, the algorithm needs to find the minimum weight edge, which takes $O(V)$ time, and this process is repeated for all V vertices.

When using an adjacency list and a min-heap (priority queue), the time complexity is $O((V + E) \log V)$, where E is the number of edges. This is because the algorithm performs V insertions and E decrease-key operations on the priority queue, each taking $O(\log V)$ time

Greedy Method

Dijkstra's Shortest Path

- Dijkstra's algorithm finds the shortest paths of a graph's vertices in order of their distance from a given source.
- First, it finds the shortest path from the source to a vertex nearest to it, then to a second nearest, and so on.

- Here, we consider the best-known algorithm for the **Single-source shortest-paths problem**, called **Dijkstra's algorithm**.
- This algorithm is applicable to **undirected and directed graphs with non-negative weights only**.
- Since in most applications this condition is satisfied, the **Dijkstra's algorithm** is popular.

Dijkstra's algorithm

Dijkstra's Algorithm allows you to calculate the shortest path between one node to every other node in the directed weighted graph.

Algorithm:

1. Mark all nodes as **unvisited**.

visit[]=False;

2. Mark the selected **initial** node with a current distance of **0** and the rest with **infinity**.

Dist[]=INF, Dist[0]=0;

3. Set the **initial** node as **current** node. **visit[0]=True;**

4. Find all adjacent node of **current** node, and calculate their distances by adding the current distance of **current** node to the weight of the edge connecting **neighbor/adjacent** node and **current** node.

Adj(0)={ say 1, 3}, Edge→{(0,1),(0,3)}

Temp1=Dist[0]+w(0,1)

Temp2=Dist[0]+w(0,3)

5. Compare the newly calculated distance to the current distance assigned to the **neighboring** node and set it as the **new** current distance of **neighboring** node (if it is less than existing).

If(Temp1 < Dist[1]) then
 Dist[1]= Temp1

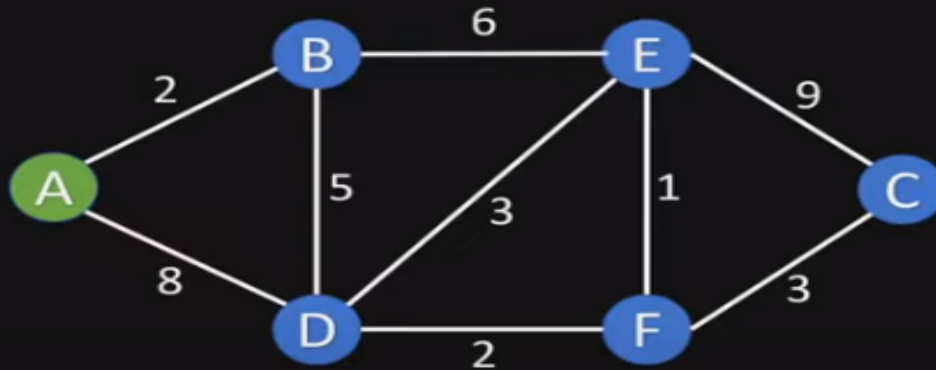
If(Temp2 < Dist[1]) then
 Dist[1]= Temp2

6. Mark the **current** node as **visited**.

Visit[1]=True

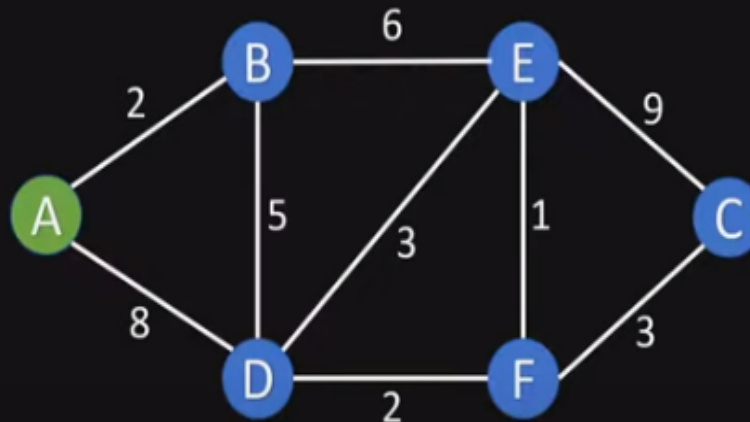
7. If the **destination** node has been marked **visited** then stop. The algorithm has finished.
8. Otherwise, select the **unvisited** node that is marked with the **smallest** distance, set it as the new **current node**, and go back to **step 4**.

Example



Visited Nodes: []

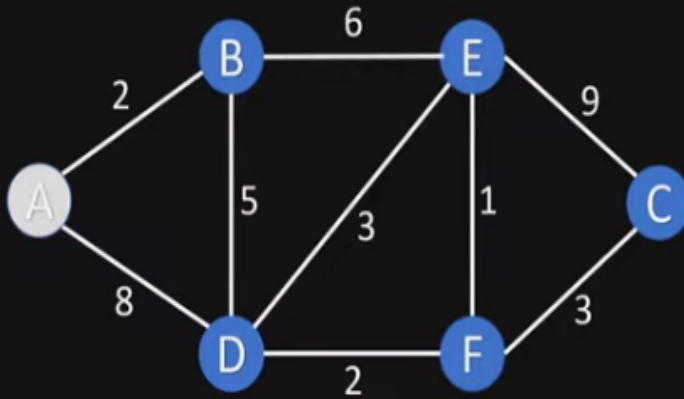
Unvisited Nodes: [A, B, C, D, E, F]



Visited Nodes: []

Unvisited Nodes: [A, B, C, D, E, F]

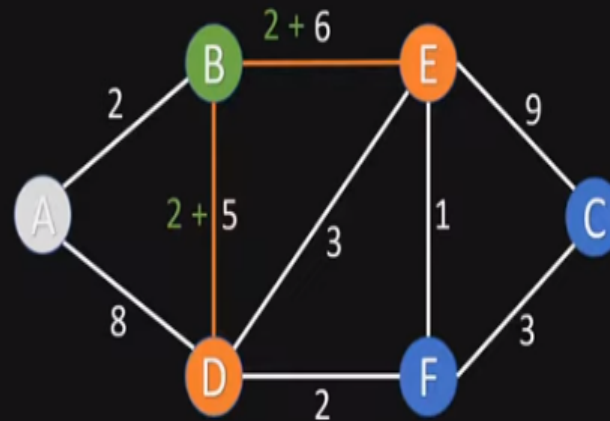
Node	Shortest Distance	Previous Node
A	0	
B	∞	
C	∞	
D	∞	
E	∞	
F	∞	



Visited Nodes: [A]

Unvisited Nodes: [B, C, D, E, F]

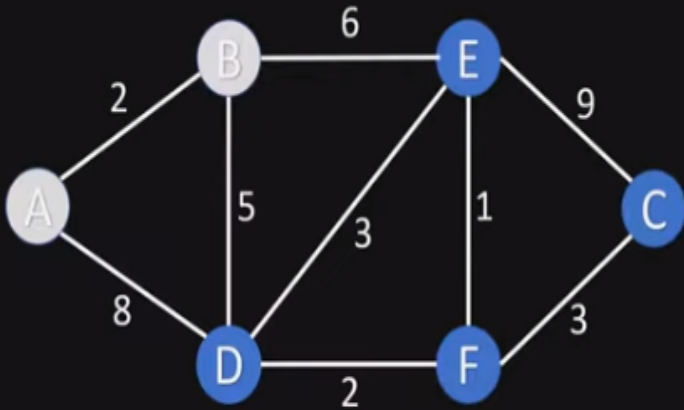
Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	8	A
E	∞	
F	∞	



Visited Nodes: [A]

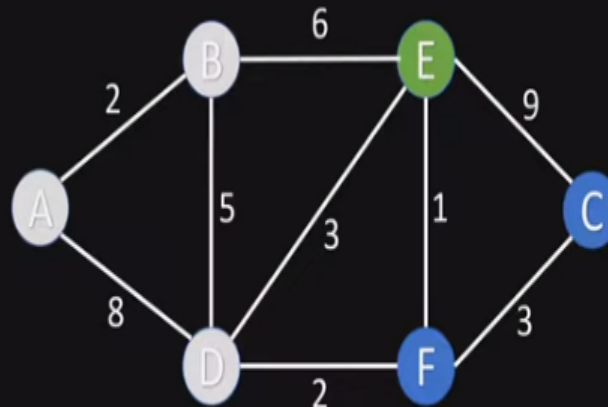
Unvisited Nodes: [B, C, D, E, F]

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	8	A
E	∞	
F	∞	



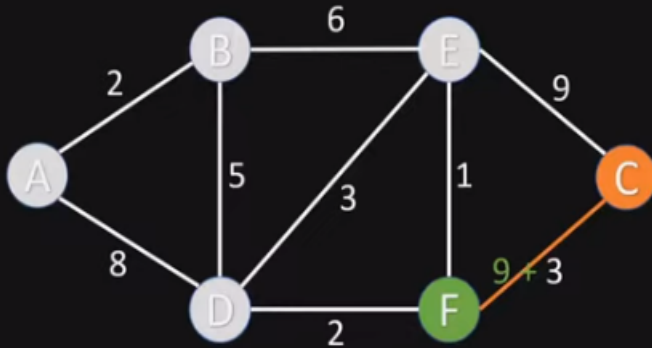
Visited Nodes: [A, B] Unvisited Nodes: [C, D, E, F]

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	7	B
E	8	B
F	∞	



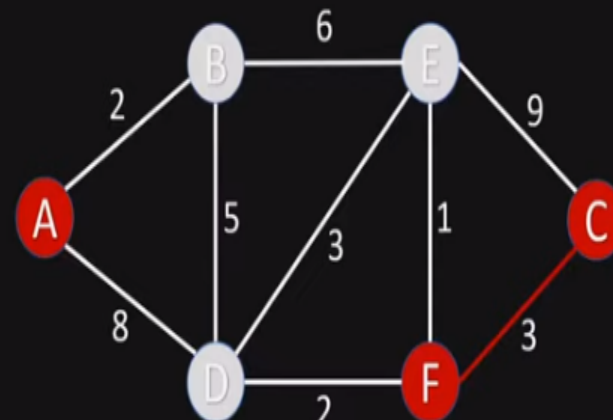
Visited Nodes: [A, B, D] Unvisited Nodes: [C, E, F]

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	7	B
E	8	B
F	9	D



Visited Nodes: [A, B, D, E] Unvisited Nodes: [C, F]

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	17	E
D	7	B
E	8	B
F	9	D

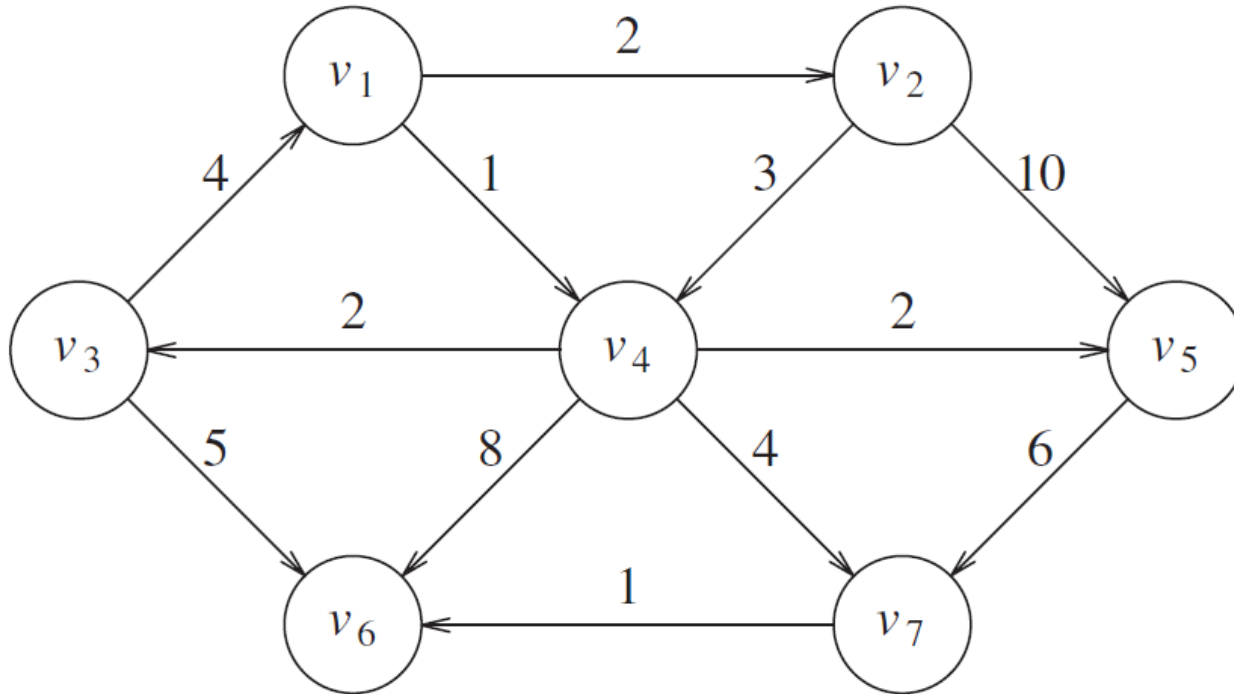


Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

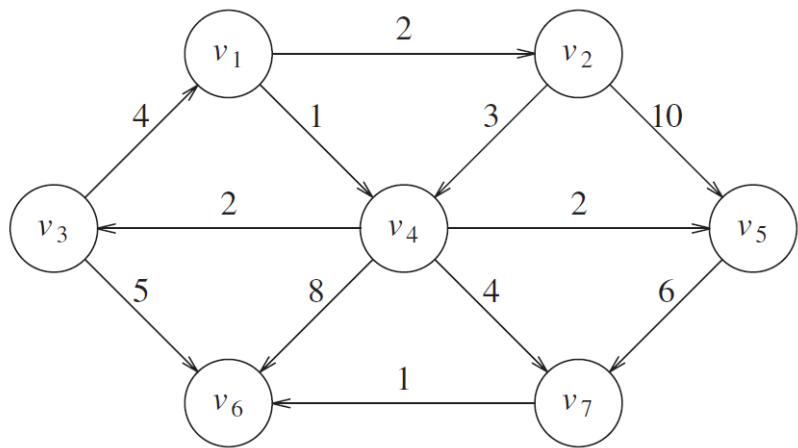
Introduction to Dijkstra's algorithm



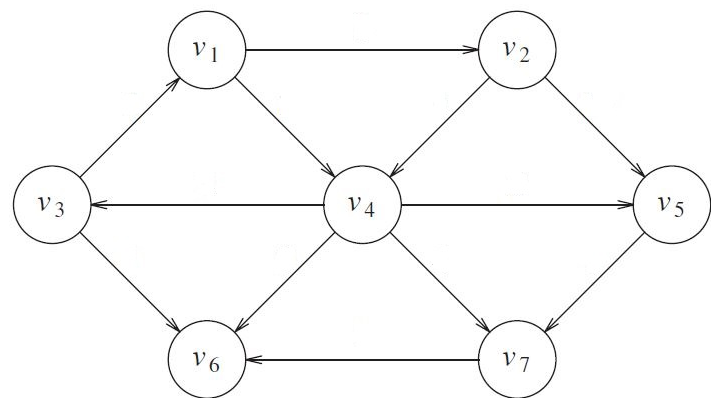
Find the Shortest path from the following directed Graph, starting node is V1



Dijkstra's algorithm Problem



Initial directed Graph

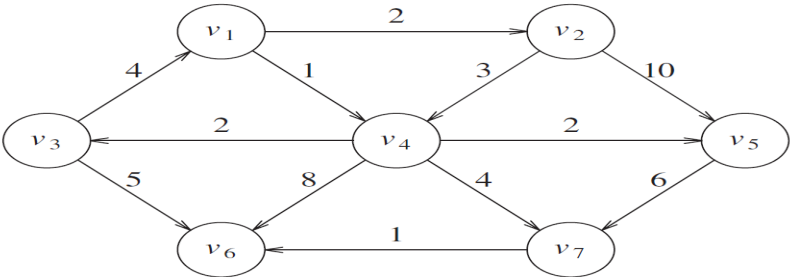


Starring node is v1

v	$known$	d_v	p_v
v_1	F	0	0
v_2	F	∞	0
v_3	F	∞	0
v_4	F	∞	0
v_5	F	∞	0
v_6	F	∞	0
v_7	F	∞	0

Initial path table

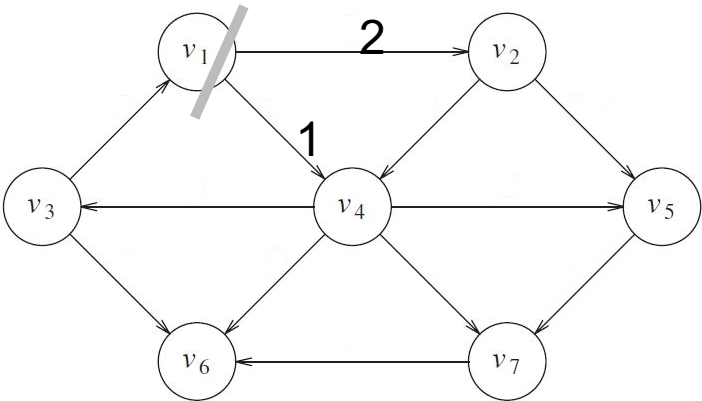
Dijkstra's algorithm Problem



Step1: v1 is known

Directed Graph

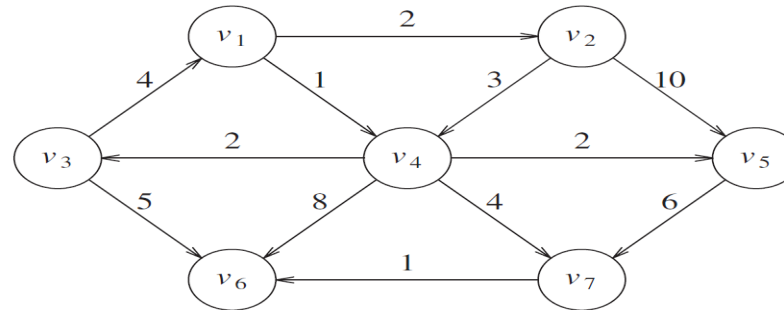
Find the adjacent nodes of (v1)={ v2, v4 }



Path table

<i>v</i>	<i>known</i>	<i>d_v</i>	<i>p_v</i>
v ₁	T	0	0
v ₂	F	2	v ₁
v ₃	F	∞	0
v ₄	F	1	v ₁
v ₅	F	∞	0
v ₆	F	∞	0
v ₇	F	∞	0

Dijkstra's algorithm Problem

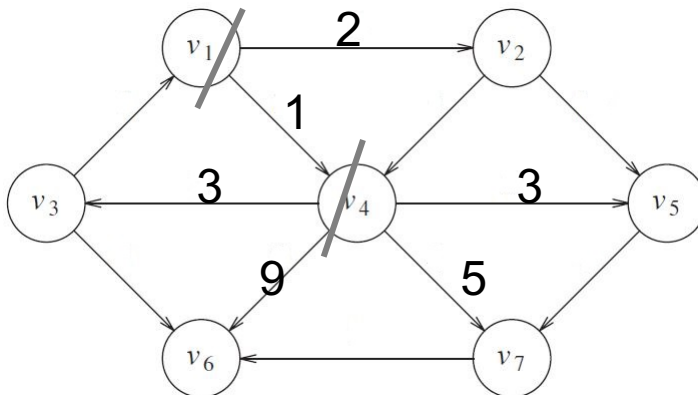


Step2: v_4 is declared known

Directed Graph

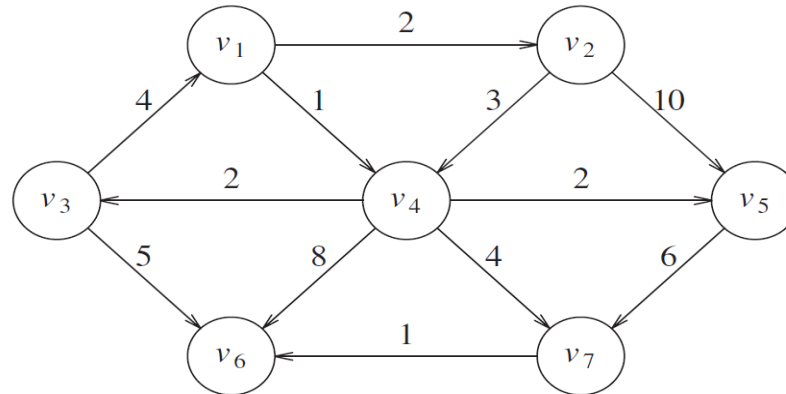
Find the adjacent nodes of $(v_4) = \{v_3, v_6, v_7, v_5\}$
 $\{2, 8, 4, 2\}$

Path table



v	$known$	d_v	p_v
v_1	T	0	0
v_2	F	2	v_1
v_3	F	3	v_4
v_4	T	1	v_1
v_5	F	3	v_4
v_6	F	9	v_4
v_7	F	5	v_4

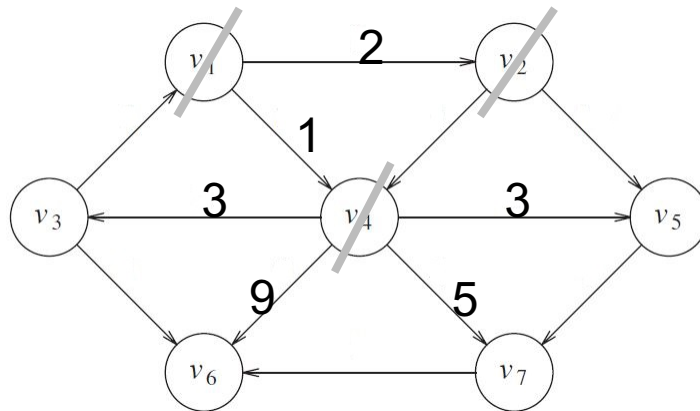
Dijkstra's algorithm Problem



Step3: v2 is declared known

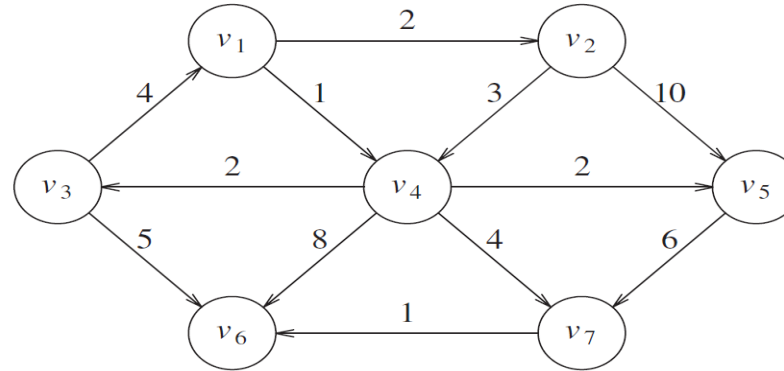
Directed Graph

Adjacent nodes of (v2)={ v5, v4 }



v	$known$	d_v	p_v
v_1	T	0	0
v_2	T	2	v_1
v_3	F	3	v_4
v_4	T	1	v_1
v_5	F	3	v_4
v_6	F	9	v_4
v_7	F	5	v_4

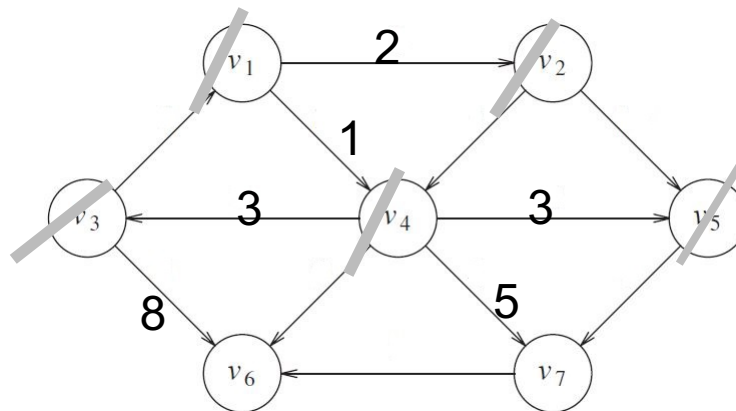
Dijkstra's algorithm Problem



Step4: v5 and then v3 are declared known

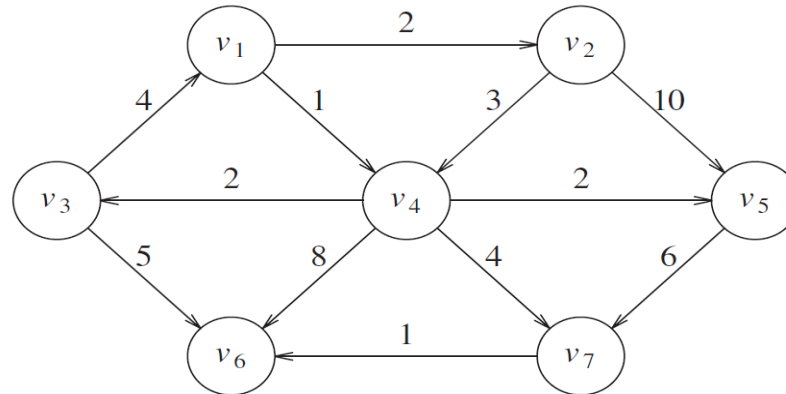
Path table

Adjacent nodes of (v3)={ v1, v6 }



v	$known$	d_v	p_v
v_1	T	0	0
v_2	T	2	v_1
v_3	T	3	v_4
v_4	T	1	v_1
v_5	T	3	v_4
v_6	F	8	v_3
v_7	F	5	v_4

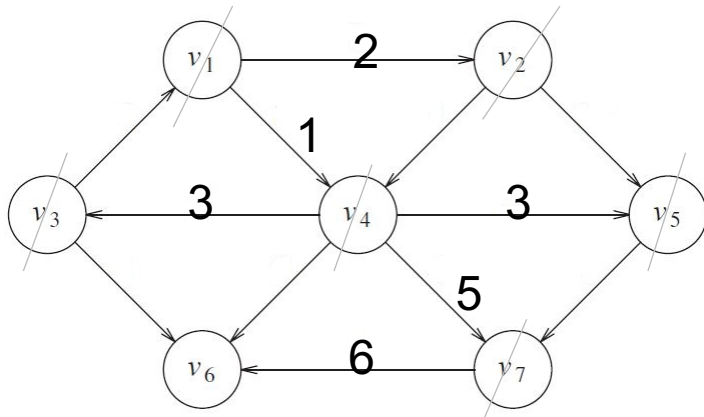
Dijkstra's algorithm Problem



Path table

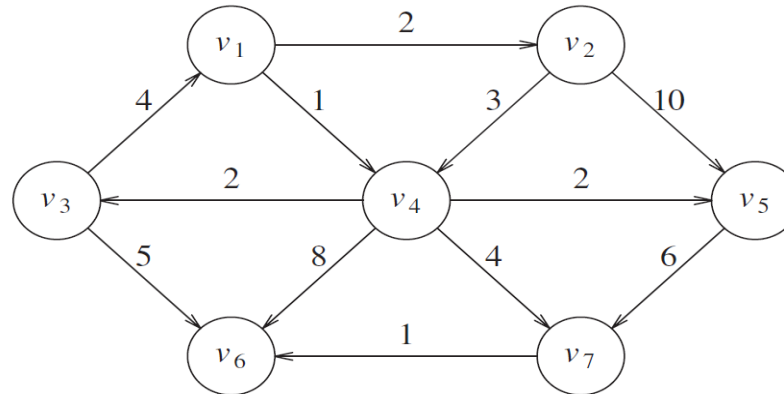
Step5: v7 is declared known

Adjacent nodes of (v7)={ v6, v5 }



v	$known$	d_v	p_v
v_1	T	0	0
v_2	T	2	v_1
v_3	T	3	v_4
v_4	T	1	v_1
v_5	T	3	v_4
v_6	F	6	v_7
v_7	T	5	v_4

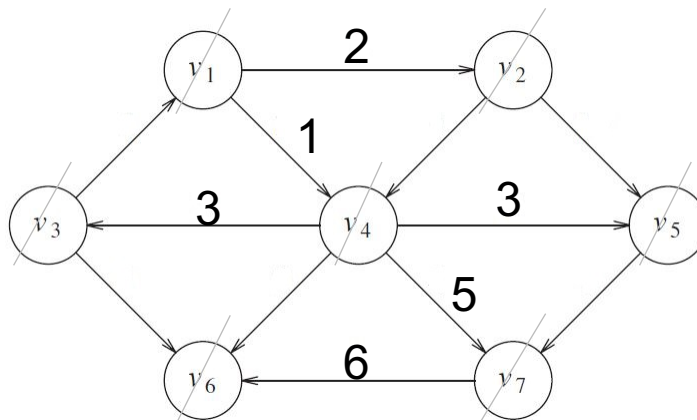
Dijkstra's algorithm Problem



Step6: v6 is declared known and algorithm terminates

Adjacent nodes of (v6)={ }

Path table



v	$known$	d_v	p_v
v_1	T	0	0
v_2	T	2	v_1
v_3	T	3	v_4
v_4	T	1	v_1
v_5	T	3	v_4
v_6	T	6	v_7
v_7	T	5	v_4

Dijkstra's algorithm Problem



Result: Distance pairs

$$(v_1, v_2) = 2$$

$$(v_4, v_3) = 3$$

$$(v_1, v_4) = 1$$

$$(v_4, v_5) = 3$$

$$(v_7, v_6) = 6$$

$$(v_4, v_7) = 5$$

Total cost in Shortest path is = 20

v	$known$	d_v	p_v
v_1	T	0	0
v_2	T	2	v_1
v_3	T	3	v_4
v_4	T	1	v_1
v_5	T	3	v_4
v_6	T	6	v_7
v_7	T	5	v_4

ALGORITHM *Dijkstra(G, s)*

//Dijkstra's algorithm for single-source shortest paths

//Input: A weighted connected graph $G = \langle V, E \rangle$ with nonnegative weights

// and its vertex s

//Output: The length d_v of a shortest path from s to v

Initialize(Q) //initialize priority queue to empty

for every vertex v in V

$d_v \leftarrow \infty$; $p_v \leftarrow \mathbf{null}$

Insert(Q, v, d_v) //initialize vertex priority in the priority queue

$d_s \leftarrow 0$; *Decrease(Q, s, d_s)* //update priority of s with d_s

$V_T \leftarrow \emptyset$

for $i \leftarrow 0$ **to** $|V| - 1$ **do**

$u^* \leftarrow \text{DeleteMin}(Q)$ //delete the minimum priority element

$V_T \leftarrow V_T \cup \{u^*\}$

for every vertex u in $V - V_T$ that is adjacent to u^* **do**

if $d_{u^*} + w(u^*, u) < d_u$

$d_u \leftarrow d_{u^*} + w(u^*, u)$; $p_u \leftarrow u^*$

Decrease(Q, u, d_u)

V_T

V d p

1	INF	null
2	INF	null
3	INF	null
4	INF	null
5	INF	null
6	INF	null

```
class Dijkstra
```

```
{  public static void main(String[] args)
    {  int V=8; int G[][] = new int[][] { {0,14,0,0,2,0,0,8,0},
                                             {4,0,8,0,0,0,0, 11,0},
                                             {0,2,0,0,0,8,0,0,2},
                                             {0,3,0,0,9,7,0,9,0},
                                             {0,0,0,9,0,10,0,0,0},
                                             {0,0,9,4,0,0,7,0,0},
                                             {0,0,8,0,0,2,0,11,0},
                                             {0,9,0,0,0,0,8,0,3},  };
}
```

```
int dist[] = new int[V];
```

```
boolean visited[]=new boolean[V];
```

Dijkstra's Java Program

```
for (int i = 0; i < V; i++)  
{  
    dist[i] = 99999;  //---set to infinity  
    visited[i] = false;  //--- not visited  
}
```

//---Find a node(v) with min distance in V[] and not in V_T[]-----

```
dist[0] = 0;
```

```
for (int c=0;c<V-1;c++)
```

```
{    int min = 99999, minIdx = -1;
```

```
    for (int v = 0; v < V; v++)
```

```
        if (visited[v] == false && dist[v] <= min)
```

```
        {    min = dist[v];
```

```
            minIdx = v;
```

```
        }
```

```
    visited[minIdx] = true;
```

Min Priority
Queue

```
//-----Find adjacent(v) and update min distance -----  
for (int v = 0; v < V; v++)  
{  if (!visited[v] && G[minIdx][v] != 0 && dist[minIdx] != 99999)  
    if(dist[minIdx] + G[minIdx][v] < dist[v])  
        dist[v] = dist[minIdx] + G[minIdx][v];  
}  
}  
for (int i = 0; i < V; i++)  
System.out.println("0 " + " to " + i + " Min Distance is " + dist[i]);  
}  
}
```

- If the input graph is implemented using adjacency matrix, it is **$O(V^2)$** .
- If the input graph is represented using adjacency list, it can be reduced to **$O(E \log V)$** with the help of binary heap (priority queue).

String Matching Algorithm

(Naive string matching, Rabin-Karp algorithm)

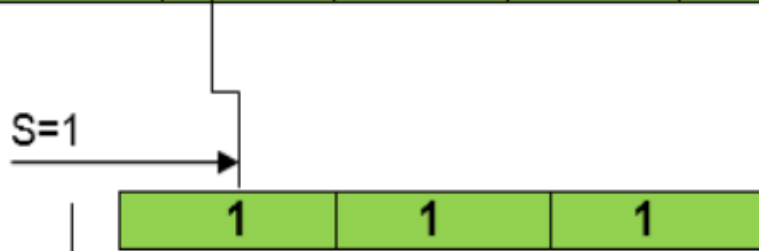
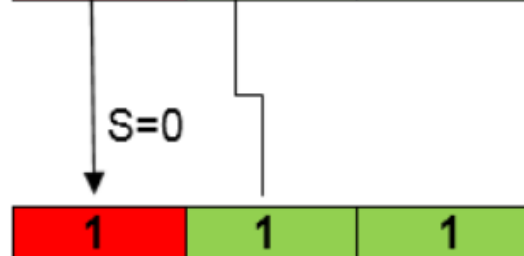
- String-matching algorithm, is an algorithm that searches a body of text for portions that match by pattern.
- Example: Text: 11010101110010111
–Pattern: 1110

- The naive approach tests all the possible placement of Pattern $P [1.....m]$ relative to text $T [1.....n]$.
- We try shift $s = 0, 1.....n-m$, successively and for each shift s . Compare $T [s+1.....s+m]$ to $P [1.....m]$.
- The naive algorithm finds **all valid shifts using a loop that checks the condition $P [1.....m] = T [s+1.....s+m]$** for each of the $n - m + 1$ possible value of s .

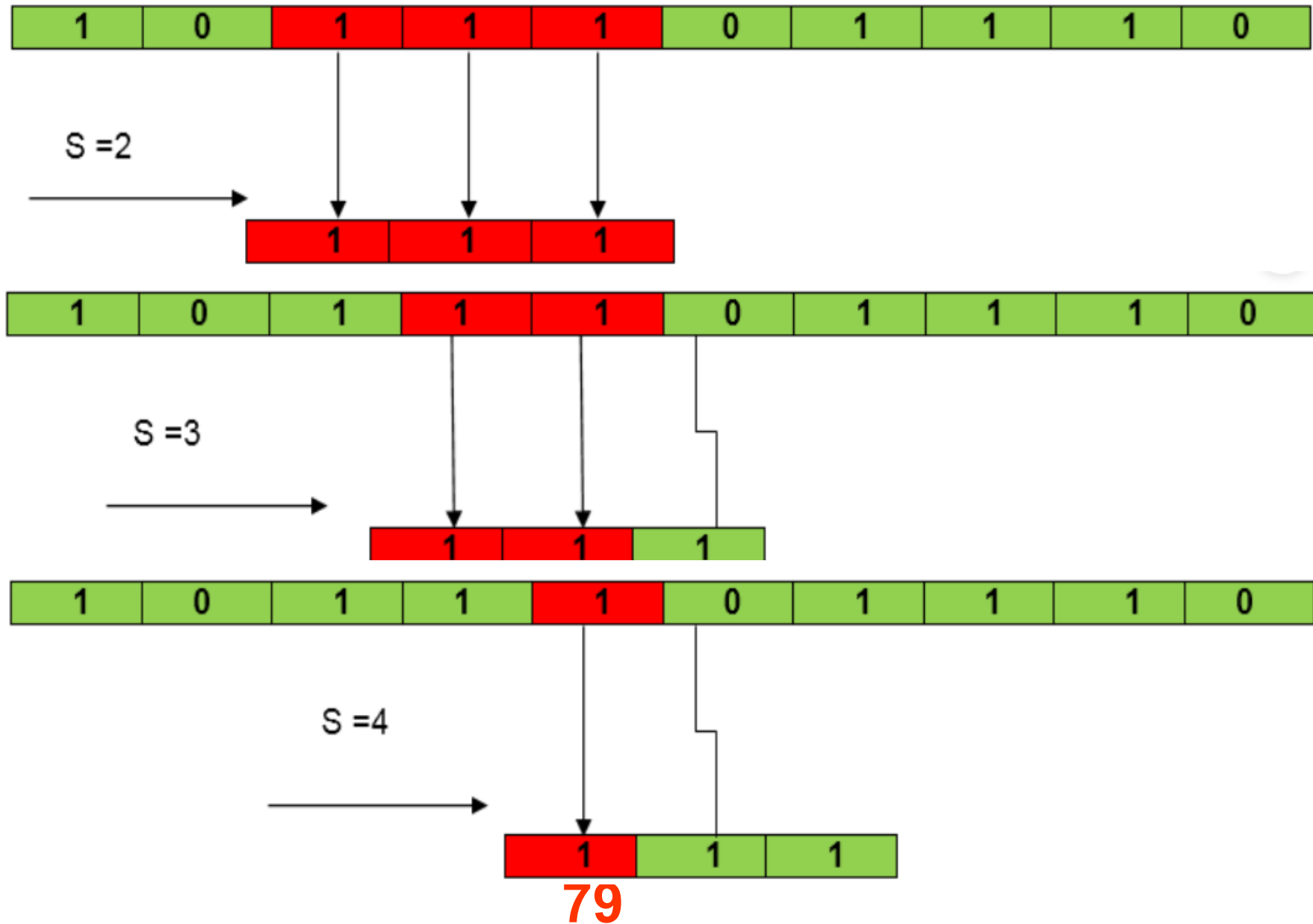
Example

For Example, $T = 1011101110$ & $P = 111$

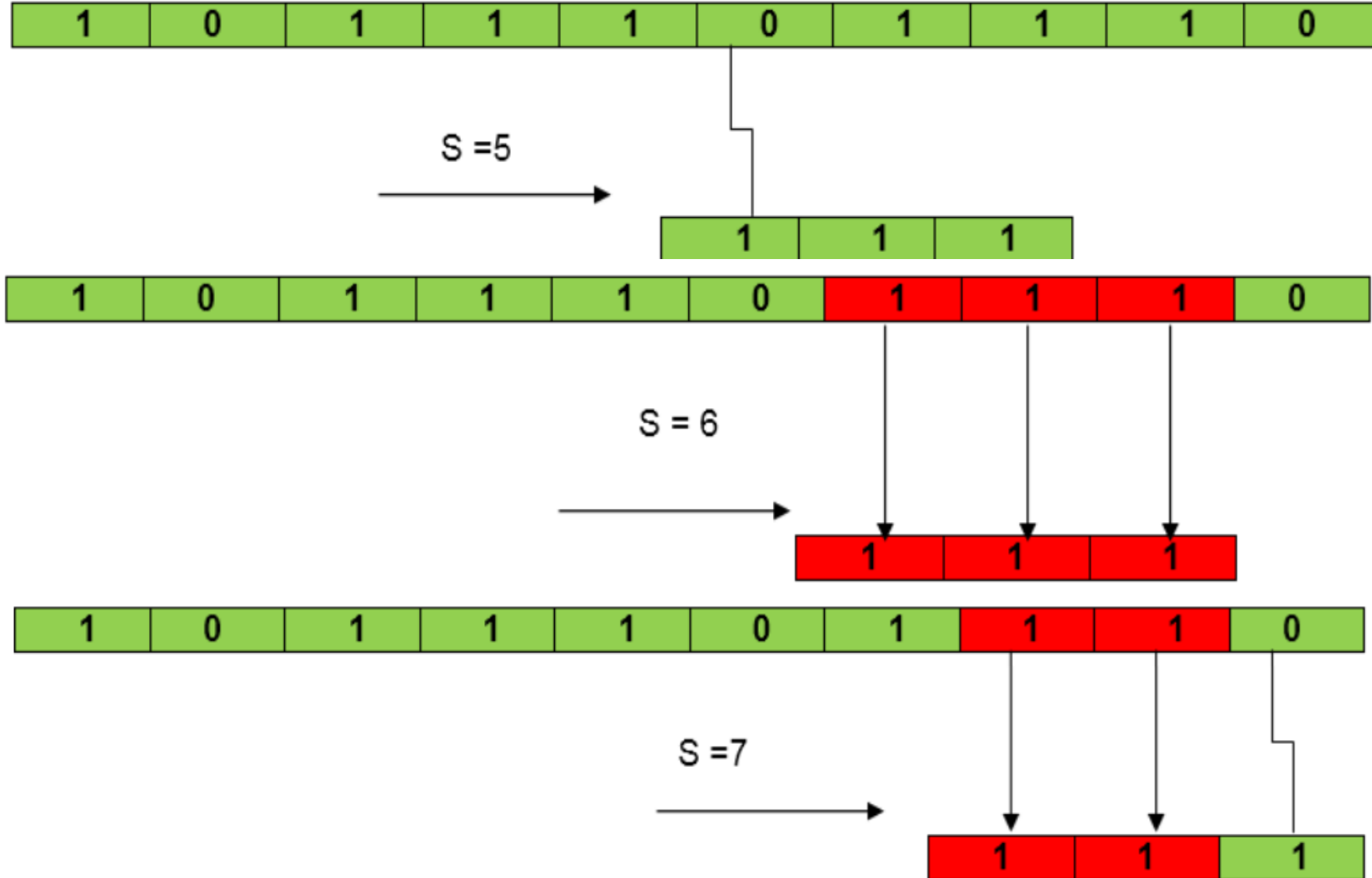
Find all the Valid Shift



VIT AP UNIVERSITY, Andhra Pradesh



Continue..



NAIVE-STRING-MATCHER (T, P)

```
1   $n = T.length$ 
2   $m = P.length$ 
3  for  $s = 0$  to  $n - m$ 
4      if  $P[1..m] == T[s + 1..s + m]$ 
5          print "Pattern occurs with shift"  $s$ 
```

NAIVE-STRING-MATCHER takes time $O((n - m + 1) * m)$

- The Rabin-Karp algorithm is a popular string matching algorithm that utilizes hashing function/technique.
- It compares the hash values of the pattern and each substring of the text to determine potential matches.
- This algorithm is particularly useful in applications like searching documents, detecting plagiarism, and analyzing DNA sequences

Rabin-Karp algorithm

RABIN-KARP-MATCHER(T, P, d, q)

A prime number for mod operation

Length of input text, in case of all alphabets 26

```
1   $n = T.length$ 
2   $m = P.length$ 
3   $h = d^{m-1} \bmod q$ 
4   $p = 0$ 
5   $t_0 = 0$ 
6  for  $i = 1$  to  $m$                                 // preprocessing
7       $p = (dp + P[i]) \bmod q$ 
8       $t_0 = (dt_0 + T[i]) \bmod q$ 
9  for  $s = 0$  to  $n - m$                                 // matching
10     if  $p == t_s$ 
11         if  $P[1..m] == T[s + 1..s + m]$ 
12             print "Pattern occurs with shift"  $s$ 
13     if  $s < n - m$ 
14          $t_{s+1} = (d(t_s - T[s + 1]h) + T[s + m + 1]) \bmod q$ 
```

Example

$T = \{a\ b\ d\ a\ b\ e\ a\ f\ j\ c\ a\ g\ b\ h\ a\ i\ a\ j\ c\ e\}$, $n=20$

$P = \{j\ c\ a\}$ $m=3$, $d=10$ {a to j alphabets only used} $q=7$

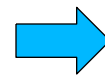
a	b	d	a	b	e	a	f	j	c	a	g	b	h	a	i	a	j	c	e
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

j	c	a
---	---	---

$$h = d^{m-1} \bmod q$$

$$p = 0$$

$$t_0 = 0$$



$$h = 10^{(m-1)} \% q$$

$$h = 10^2 \% 7 = 2$$

- If we take ascii value of characters, the calculated value may exceed the integer range.
- To avoid this we can assign smaller number to all alphabets like $a=1$, $b=2$ to $z=26$.
- In case of big text / pattern it may exceed the integer range, so mod operation is used.

a	b	d	a	b	e	a	f	j	c	a	g	b	h	a	i	a	j	c	e
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

for $i = 1$ to m

// preprocessing

j	c	a
---	---	---

$$p = (dp + P[i]) \bmod q$$

$$t_0 = (dt_0 + T[i]) \bmod q$$

$$h = d^{m-1} \bmod q$$

$$p = 0$$

$$t_0 = 0$$

- $i=1$, $p=(d*p+P[i])\%7 \rightarrow (10*0+10)\%7=3$
 $t_0=(d*t_0+T[i])\%7 \rightarrow (10*0+1)\%7=1$, now $p=3$, $t_0=1$
- $i=2$, $p=(d*p+P[i])\%7 \rightarrow (10*3+3)\%7=5$
 $t_0=(d*t_0+T[i])\%7 \rightarrow (10*1+2)\%7=5$, now $p=5$, $t_0=5$
- $i=3$, $p=(d*p+P[i])\%7 \rightarrow (10*5+1)\%7=2$
 $t_0=(d*t_0+T[i])\%7 \rightarrow (10*5+4)\%7=5$, now $p=2$, $t_0=5$

if($p == t_0$) then

Match individual characters, move to next shift ($t_s=t_{s+1}$)

Else

$t_s=t_{s+1}$ repeat the process upto $(n-m)+1$ shifts

Example...



- Let $T = \{a \ b \ a \ c \ d \ b \ d \ e\}$, $n=8$ $P = \{b \ a \ c\}$, $m=30$
– $\{1 \ 2 \ 1 \ 3 \ 4 \ 2 \ 4 \ 5\}$
- $P=1$

- RKM takes $\Theta(m)$ preprocessing time, and its matching time is $\Theta((n-m+1)m)$ in the worst case, since (like the naive string-matching algorithm) the Rabin-Karp algorithm explicitly verifies every valid shift.
- In most applications, we expect few valid shifts, assume some constant 'c' of them. In such applications, the expected matching time of the algorithm is only.
- $O((n-m+1) + cm) = O(n + m) = O(n)$

Dynamic Programming

- It was invented by a prominent U.S. mathematician, **Richard Bellman**, in the 1950s as a general method for optimizing multistage decision processes.
- Thus, the word “**programming**” in the name of this technique stands for “planning” and does not refer to computer programming.
- Dynamic programming is typically applied to optimization problems.

- Dynamic programming, like the divide-and-conquer method, solves problems by combining the solutions to subproblems.
- In contrast, dynamic programming is applicable **when the subproblems are not independent (overlapping)**, that is, when subproblems share subsubproblems. **[Example: Fibonacci Numbers]**
- It solves **every subsubproblem just once** and then **saves its answer in a table**, thereby avoiding the work of recomputing the answer every time the subsubproblem is encountered.

- Difference between the greedy method and dynamic programming:
 - In the greedy method **only one decision sequence** is ever generated.
 - In dynamic programming, **many decision sequences** may be generated.
 - However, sequences containing suboptimal subsequences cannot be optimal (if the principle of optimality holds) and so will not (as far as possible) be generated.

All-Pairs Shortest-Paths Problem

- Let $G = (V, E)$ be a directed graph with n vertices.
- Let $cost$ be a cost adjacency matrix for G such that
 - $cost(i, i) = 0, 1 \leq i \leq n$ and
 - $cost(i, j)$ is the length/cost of edge (i, j) if $(i, j) \in E(G)$ and $cost(i, j) = \infty$ if $i \neq j$ and $(i, j) \notin E(G)$.
- The **all-pairs shortest-path** problem is to determine a matrix A such that $A(i, j)$ is the length of a shortest path from i to j .

- All-pairs shortest-paths problem on a directed graph $G = (V, E)$.

- **Stephen Warshall's algorithm**— for computing the transitive closure of a directed graph.

The **transitive closure** of a graph is a way to determine if there is a path between any two vertices in a directed graph.

- **Robert W. Floyd's algorithm**— for the all-pairs shortest-paths problem.

- The **transitive closure** of a directed graph with n vertices can be defined as the $n \times n$ Boolean matrix
- $T = \{t_{ij}\}$, in which the element in the i^{th} row and the j^{th} column is **1 if there exists a nontrivial path** (i.e., directed path of a positive length) from the i^{th} vertex to the j^{th} vertex; otherwise, t_{ij} is 0.

ALGORITHM *Warshall*($A[1..n, 1..n]$)

//Implements Warshall's algorithm for computing the transitive closure

//Input: The adjacency matrix A of a digraph with n vertices

//Output: The transitive closure of the digraph

$R^{(0)} \leftarrow A$

for $k \leftarrow 1$ **to** n **do**

for $i \leftarrow 1$ **to** n **do**

for $j \leftarrow 1$ **to** n **do**

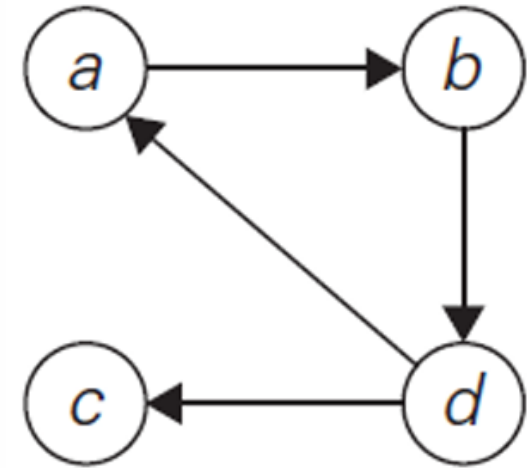
$R^{(k)}[i, j] \leftarrow R^{(k-1)}[i, j] \text{ or } (R^{(k-1)}[i, k] \text{ and } R^{(k-1)}[k, j])$

return $R^{(n)}$

Time Complexity $\Theta(n^3)$

```
Warshall(boolean G[][],int n) {  
    for(int k=0;k<n;k++)  
        { for(int i=0;i<n;i++)  
            { for(int j=0;j<n;j++)  
                {  
                    G[i][j]=G[i][j]||(G[i][k]&&G[k][j]);  
                }  
            }  
        }  
}
```

■ Example

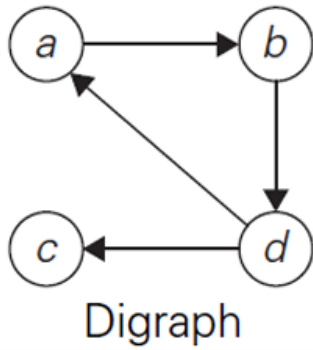


Digraph

$$A = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

$$T = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

Example



$$R^{(0)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

$$R^{(1)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 \end{bmatrix} \end{matrix}$$

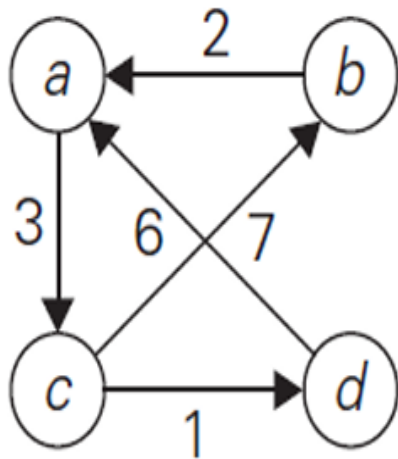
$$R^{(2)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

$$R^{(3)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

$$R^{(4)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

Floyd's Algorithm

- Given a weighted connected graph (undirected or directed), the all-pairs shortest-paths problem asks to find the distances—i.e., the lengths of the shortest paths—from each vertex to all other vertices.



$$W = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \infty & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{bmatrix} \end{matrix}$$

$$D = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 10 & 3 & 4 \\ 2 & 0 & 5 & 6 \\ 7 & 7 & 0 & 1 \\ 6 & 16 & 9 & 0 \end{bmatrix} \end{matrix}$$

ALGORITHM *Floyd*($W[1..n, 1..n]$)

//Implements Floyd's algorithm for the all-pairs shortest-paths problem

//Input: The weight matrix W of a graph with no negative-length cycle

//Output: The distance matrix of the shortest paths' lengths

$D \leftarrow W$ //is not necessary if W can be overwritten

for $k \leftarrow 1$ **to** n **do**

for $i \leftarrow 1$ **to** n **do**

for $j \leftarrow 1$ **to** n **do**

$D[i, j] \leftarrow \min\{D[i, j], D[i, k] + D[k, j]\}$

return D

Time Complexity $\Theta(n^3)$

Floyd-Warshall Algorithm

```
0  Algorithm AllPaths(cost, A, n)
1  // cost[1 : n, 1 : n] is the cost adjacency matrix of
2  // a graph with n vertices; A[i, j] is the cost of
3  // a shortest path from vertex i to vertex j.
4  // cost[i, i] = 0.0, for  $1 \leq i \leq n$ .
5  {
6      for i := 1 to n do
7          for j := 1 to n do
8              A[i, j] := cost[i, j]; // Copy cost into A.
9          for k := 1 to n do
10             for i := 1 to n do
11                 for j := 1 to n do
12                     A[i, j] := min(A[i, j], A[i, k] + A[k, j]);
13 }
```

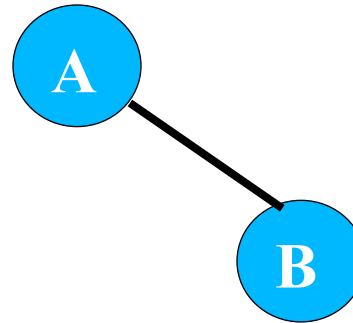
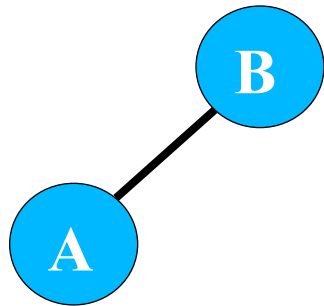
Book: Sahni

Dynamic Programming Optimal Binary Search Trees

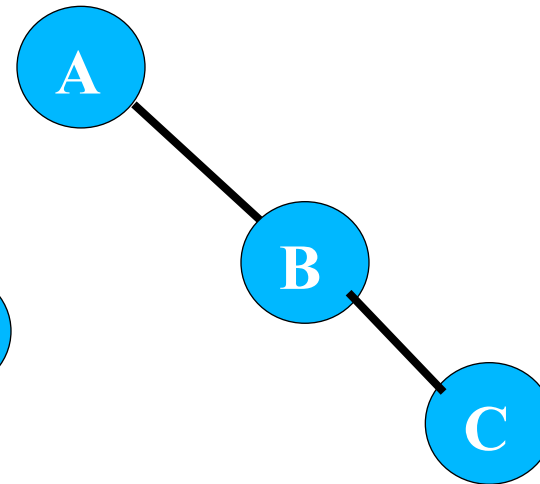
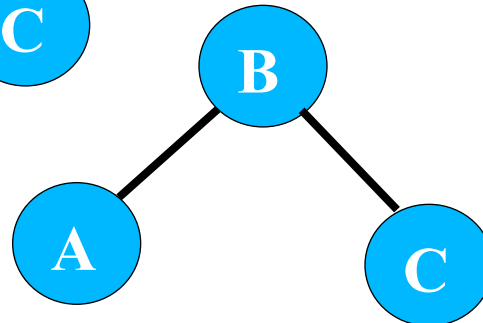
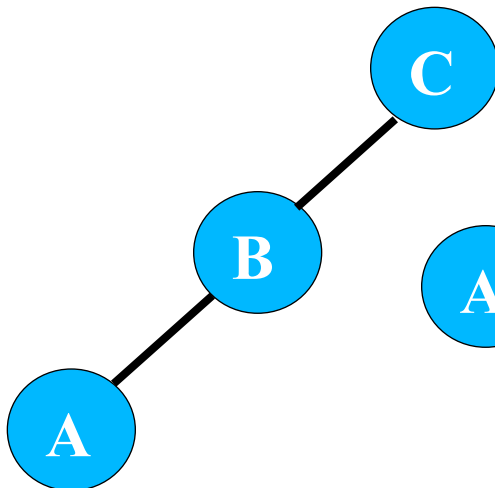
Binary Search Tree (BST)

- A binary tree with
 - all keys in left subtree are smaller than root's key
 - all keys in right subtree are larger than root's key
- One of its principal applications is to implement a dictionary
- a set of elements with the operations of searching, insertion, and deletion

- A Binary search tree with two nodes



A Binary search tree with three nodes



- Total no of Binary Trees with n nodes ?

$$t(n) = \sum_{i=1}^n t(i-1) t(n-i)$$

$$\frac{(2n)!}{(n+1)!n!}$$

- The base case is $t(0) = 1$ and $t(1) = 1$.

$$t(2) = t(0)t(1) + t(1)t(0) = 2$$

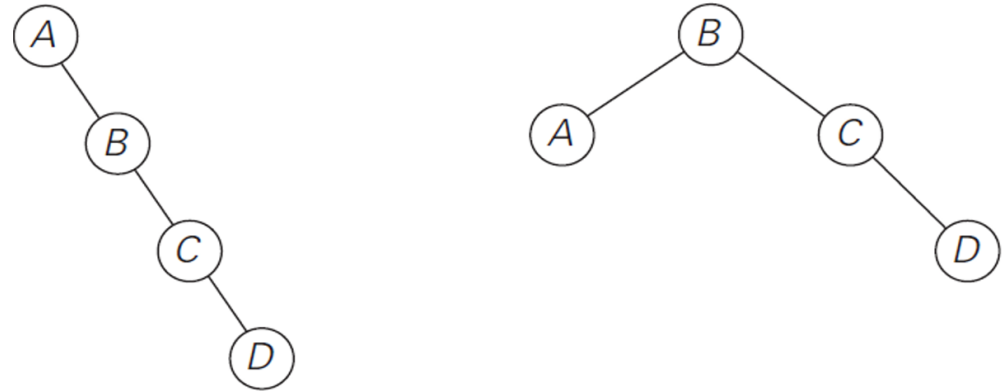
$$t(3) = t(0)t(2) + t(1)t(1) + t(2)t(0) = 2 + 1 + 2 = 5$$

$$t(4) = t(0)t(3) + t(1)t(2) + t(2)t(1) + t(3)t(0) = 5 + 2 + 2 + 5 = 14$$

- Optimal binary search tree for which the average number of comparisons in a search is the smallest possible.
- i.e In which order/way to construct a BST so that number of comparisons can be minimized for a search operation.

Example of Optimal BST

Example: Consider four keys A, B, C, and D to be searched for with probabilities 0.1, 0.2, 0.4, and 0.3, respectively.



The average number of comparisons in a successful search in first of these trees is $0.1 * 1 + 0.2 * 2 + 0.4 * 3 + 0.3 * 4 = 2.9$, and for second one it is $0.1 * 2 + 0.2 * 1 + 0.4 * 2 + 0.3 * 3 = 2.1$.

■ **Neither of these two trees is, in fact, optimal.**

For a small example, we could find the optimal tree by generating all **14 binary** search trees with these keys.

■ As a general algorithm, this exhaustive-search approach is unrealistic:

The total number of binary search trees with n keys is equal to the n th Catalan number,

If we count tree levels starting with 1 to make the comparison numbers equal the keys' levels, the following recurrence relation is obtained:

$$C(i, j) = \min_{i \leq k \leq j} \{C(i, k-1) + C(k+1, j)\} + \sum_{s=i}^j p_s \quad \text{for } 1 \leq i \leq j \leq n.$$

We assume that $C(i, i-1) = 0$ for $1 \leq i \leq n+1$, which can be interpreted as the number of comparisons in the empty tree.

$$C(i, i) = p_i \quad \text{for } 1 \leq i \leq n,$$

as it should be for a one-node binary search tree containing a_i .

ALGORITHM *OptimalBST*($P[1..n]$)

//Finds an optimal binary search tree by dynamic programming

//Input: An array $P[1..n]$ of search probabilities for a sorted list of n keys

//Output: Average number of comparisons in successful searches in the

// optimal BST and table R of subtrees' roots in the optimal BST

for $i \leftarrow 1$ **to** n **do**

$C[i, i - 1] \leftarrow 0$

$C[i, i] \leftarrow P[i]$

$R[i, i] \leftarrow i$

$C[n + 1, n] \leftarrow 0$

for $d \leftarrow 1$ **to** $n - 1$ **do** //diagonal count

for $i \leftarrow 1$ **to** $n - d$ **do**

$j \leftarrow i + d$

$minval \leftarrow \infty$

for $k \leftarrow i$ **to** j **do**

if $C[i, k - 1] + C[k + 1, j] < minval$

$minval \leftarrow C[i, k - 1] + C[k + 1, j]; \quad kmin \leftarrow k$

$R[i, j] \leftarrow kmin$

$sum \leftarrow P[i]; \quad \textbf{for } s \leftarrow i + 1 \textbf{ to } j \textbf{ do } sum \leftarrow sum + P[s]$

$C[i, j] \leftarrow minval + sum$

return $C[1, n], R$

$C[n+2][n+1]$

if $n=4$

$R[n+2][n+1]$

	0	1	2	3	4
0					
1	0	P1			
2		0	P2		
3			0	p3	
4				0	P4
5					0

Example: STEP1



Example

key	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
probability	0.1	0.2	0.4	0.3

The initial tables look like this:

main table					
	0	1	2	3	4
1	0	0.1			
2		0	0.2		
3			0	0.4	
4				0	0.3
5					0

root table					
	0	1	2	3	4
1		1			
2			2		
3				3	
4					4
5					

Let us compute $C(1, 2)$:

$$C(1, 2) = \min \left\{ \begin{array}{l} k=1: C(1, 0) + C(2, 2) + \sum_{s=1}^2 p_s = 0 + 0.2 + 0.3 = 0.5 \\ k=2: C(1, 1) + C(3, 2) + \sum_{s=1}^2 p_s = 0.1 + 0 + 0.3 = 0.4 \end{array} \right\} = 0.4.$$

Thus, out of two possible binary trees containing the first two keys, A and B , the root of the optimal tree has index 2 (i.e., it contains B), and the average number of comparisons in a successful search in this tree is 0.4.

We will ask you to finish the computations in the exercises. You should arrive at the following final tables:

main table					
	0	1	2	3	4
1	0	0.1	0.4	1.1	1.7
2		0	0.2	0.8	1.4
3			0	0.4	1.0
4				0	0.3
5					0

root table					
	0	1	2	3	4
1		1	2	3	3
2			2	3	3
3				3	3
4					4
5					

The average number of keys comparison in the optimal BST is 1.7

$R(1,4)=3$, the root of the optimal tree contains 3rd key(i.e. **C**).

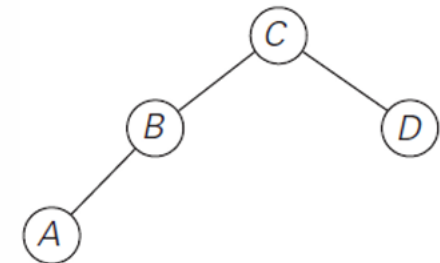
Its left sub tree is mapped as A, B and its right sub tree is D.

To find the specific structure of sub tree, we have to find the root of the sub tree, by referring root table again.

$R(1,2)=2$, so root of the subtree is 2nd key (i.e. **B**)

$R(1,1)=1$, root of the one-node tree is **A**

$R(4,4)=4$, root of the one-node tree is **D**



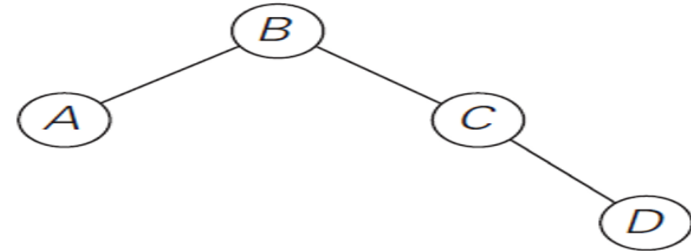
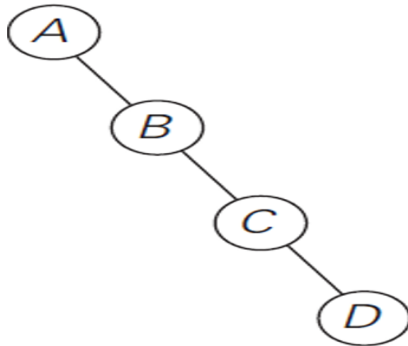
Optimal binary search tree

Optimal Binary Search Tree For Unsuccess Probability

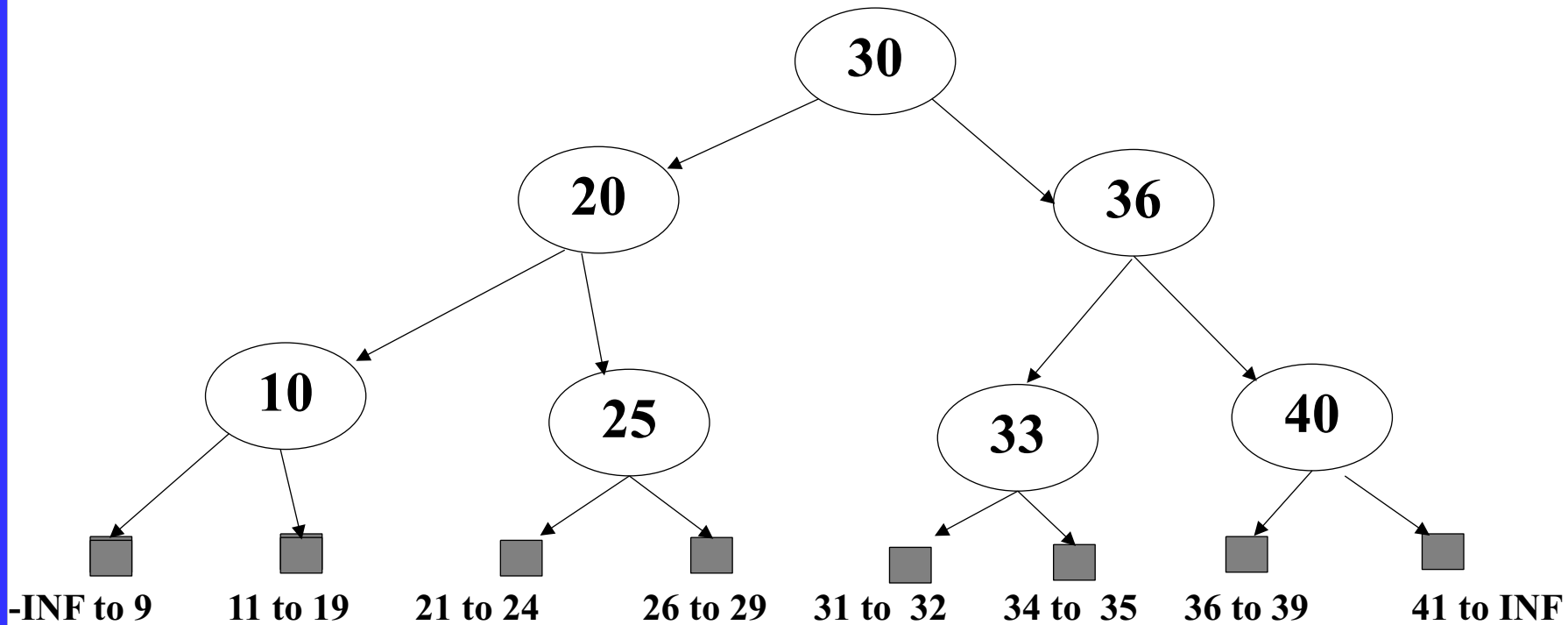
Optimal BST

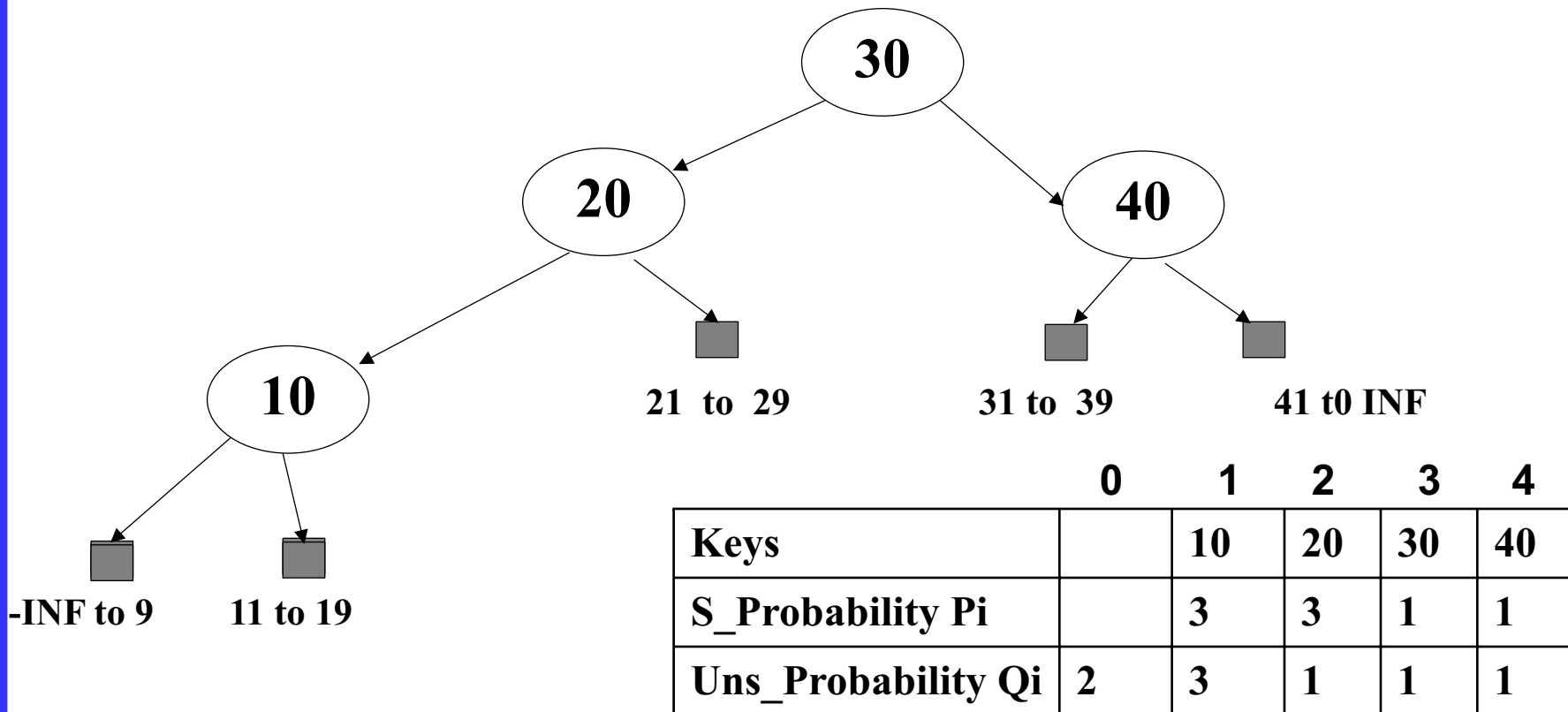


	0	1	2	3	4
Keys		10	20	30	40
S_Probability P_i		3	3	1	1
Uns_Probability Q_i	2	3	1	1	1



Example





$$\text{cost}[0,4] = P_i * \text{Level}(\text{key}) + Q_i * (\text{Level}(\text{external key}) - 1)$$

$$\text{Cost}[0,n] = \sum_{i=1}^n (P_i * \text{Level}(\text{key}) + \sum_{i=0}^n (Q_i * (\text{Level}(\text{Enteral key}) - 1))$$

$$C[i, j] = \min\{C[i, k-1] + C[k, j] + W[i, j] \quad \text{where } i < k \leq j$$

– Here, $W[i][j]$ is the sum of probabilities: $W[i][j]$
 $= \sum_{k=i}^j p[k] + \sum_{k=i-1}^j q[k]$

- $W[i, j] = W[i, j-1] + P[j] + Q[j]$

$$W[i, j] = Q[i], \quad C[i, j] = 0, \text{ if } (i == j)$$

Optimal BST



$$C[i,j] = \min\{C[i, k-1] + C[k, j] + W[i, j]\}$$

where $i < k \leq j$

$$W[i, j] = W[i, j-1] + P[j] + Q[j]$$

Keys		10	20	30	40
S_Probability P_i		3	3	1	1
Uns_Probability Q_i	2	3	1	1	1

	0	1	2	3	4
0	W=2 C=0 R=	W= C= R=	W= C= R=	W= C= R=	W= C= R=
1		W=3 C=0 R=	W= C= R=	W= C= R=	W= C= R=
2			W=1 C=0 R=	W= C= R=	W= C= R=
3				W=1 C=0 R=	W= C= R=
4					W=1 C=0 R=

Optimal BST



$$C[1,4]=32$$

$$W[0,4]=16$$

$$R[0,4]=2$$

Keys		10	20	30	40
S_Probability P_i		3	3	1	1
Uns_Probability Q_1	2	3	1	1	1

	0	1	2	3	4
0	W=2 C=0 R=0	W=8 C=8 R=1	W=12 C=19 R=1	W=14 C=25 R=2	W=16 C=32 R=2
1		W=3 C=0 R=1	W=7 C=7 R=2	W=9 C=12 R=2	W=11 C=19 R=2
2			W=1 C=0 R=2	W=3 C=3 R=3	W=5 C=8 R=3
3				W=1 C=0 R=3	W=3 C=3 R=4
4					W=1 C=0 R=4

```
Uns_Optimal_BST(P[1,n], Q[0,n])
{ C[n+1][n+1], W [n+1][n+1], R [n+1][n+1];
for(i=0;i<=n;i++)
{ W[i][i]=Q[i]; C[i][i]=0; R[i][i]=0; }
for(d=1; d<n; d++)
{ for(i=1; i<n-d; i++)
    { j=i+d; min=INF;
      for(k=i+1;k<=j; k++)
      { min=(C[i][k-1]+C[k][j] <min)?C[i][k-1]+C[k][j] :min;
        kmin=k; }
      C[i][j]=min+W[i, j-1]+P[j]+Q[j];    R[i][j]=kmin;
      W[i][j]=W[i, j-1]+P[j]+Q[j]
    }
  }
}
```

Matrix Chain Multiplication

Dynamic Programming

$$M \times N = R$$

$$[5 \times 4] \times [4 \times 3] = [5 \times 3]$$

1st row of M multiply with 1st column of N

To get $R[0][0]=4$ multiplication required

To get $R[0][1]=4$ multiplication required

Total of $[5 \times 3=15]$ elements x 4 multiplication each
i.e $5 \times 3 \times 4=60$ multiplication required.

A x B x C x D x E

[5x3] [3x4] [4x6] [6x5] [5x4]

- The objective is to not to multiply and get the answer.
- The objective is to know how to **Multiply** i.e. how to take the pair for chain of matrices / parenthesize them for multiplication so that total cost can be minimized.
- $((((A \times B) \times C) \times (D \times E)))$ or
- $(((((A \times B) \times C) \times D) \times E))$

- Like this way a total of $T(n) = (2n C n) / (n+1)$
- $T(3)=5$, we can't try all possible way to find the optimal one.
- By using dynamic programming to find the optimal parenthesize multiplication.
- Dynamic programming is a tabulation method.
- It follows bottom up approach

Algorithm Steps

A x B x C x D x E

[5x3] [3x4] [4x6] [6x5] [5x4]

$M[i,i]=0$ i.e AxA or BxB

$M[1,2]$ i.e. A x B

Cost is $\rightarrow [5x3] [3x4]=5x3x4$

$M[2,3]$ i.e. B x C

Cost is $\rightarrow [3x4] [4x6]=3x4x6=$

	1	2	3	4	5
1	0	60			
2		0	72		
3			0		
4				0	
5					0

	1	2	3	4	5
1	1				
2					
3					
4					
5					

Recurrence Relation

No. of Matrices	No. of Representation	Recurrence
(A1)	1	$T(1)=1$
(A1 A2)	1	$T(2)=1$
A1 (A2 A3) (A1 A2) A3	2	$T(3)=T(1)*T(2)+T(2)*T(1)$ $T(3)=1*1+1*1 \rightarrow 2$
A1 (A2 A3 A4) (A1 A2) (A3 A4) (A1 A2 A3) A4	5	$T(4)=T(1) T(3) + T(2) T(2) + T(3) + T(1)$ $T(4)=1*2 + 1*1 + 2*1$ $T(4)=5$
A1 (A2 A3 A4 A5) (A1 A2) (A3 A4 A5) (A1 A2 A3) (A4 A5) (A1 A2 A3 A4) A5	14	$T(5)=T(1)*T(4) + T(2)*T(3) + T(3)*T(2) + T(4)*T(1)$ $T(5)=1*5+1*2+2*1+5*1 \rightarrow 14$
$T(n)=T(1)*T(n-1)+T(2)*T(n-2)+T(3)*T(n-3)+\dots+T(n-1)*T(1)$ $T(n)=\{1, \text{ if } N \leq 2,$ $\boxed{\square} = \sum_{k=1}^{n-1} T(k) * T(n-k) \quad \text{if } n > 2$		

Recurrence Relation

$$\bullet T(n) = \frac{1}{n} * \frac{(2n-2)!}{(n-1)! * (n-1)!} \quad \text{if } n \geq 2$$

$$\bullet T(3) = \frac{1}{3} * \frac{(2*3-2)!}{(3-1)! * (3-1)!} \rightarrow 2$$

$$\bullet T(4) = \frac{1}{4} * \frac{(2*4-2)!}{(4-1)! * (4-1)!} \rightarrow 5$$

$$\bullet T(5) = \frac{1}{5} * \frac{(2*5-2)!}{(5-1)! * (5-1)!} \rightarrow 14$$

Example

- Matrices: A B C D & Product: 5, 4, 6, 2, 7
- If $C[i,j]=0$, if $i=j$
- $C[i,j] = \min\{C[i, k] + C[k+1, j] + p[i-1] * p[k] * p[j]\}$
 where $k, i \leq k < j$

1	1		
	2	2	
		3	3
			4

Sequence Table (S)

Cost Table

	1	2	3	4
1	0	120		
2		0	48	
3			0	84
4				0

1	1	1	
	2	2	3
		3	3
			4

Sequence Table (S)

Cost Table

	1	2	3	4
1	0	120	88	
2		0	48	104
3			0	84
4				0

1	1	1	3
	2	2	3
		3	3
			4

Sequence Table (S)

Cost Table

	1	2	3	4
1	0	120	88	158
2		0	48	104
3			0	84
4				0

(A B C) (D)

((A) (B C)) (D)

((A) (B C)) (D)

5x4 4x6 6x2 2x7

5x4 4x2, →
cost = 4x6x2 = 48

5x 2x7 →
cost = 5x4x2 = 40

132

5x7

cost = 5x2x7 = 70

Matrix_Chain_Multiplication(P)

```
{n=length[P]-1
for i=1 to n, do
C[i,i]=0
for L=2 to n, do
{  for i=1 to (n-L+1), do
    { j=i+L-1
      C[i, j]=Infinity
      for k=i to j-1, do
      {  q=C[i, k]+C[k+1, j]+Pi-1 Pk Pj
        if(q<C[i, j]) then
          {C[i, j]=q;  S[i, j]=k}
        }
      }
    }
  }return C & s }
```

- Given two strings, **s1** and **s2**, the task is to find the length of the Longest Common Subsequence.
- If there is no **common subsequence**, return 0.
- A **subsequence** is a sequence derived by deleting some or no elements from the original sequence without changing the order of the remaining elements.
- A **common subsequence** means a subsequence that appears in both sequences in the same relative order.

- Input: Str1 = "abcde"
- Str2 = "ace"
- Output: 3
- **Explanation:** The longest common subsequence is "**ace**" and its length is 3.
- Input: str1 = "ABC", str2 = "CBA"
- Output: 1
- **Explanation:** There are three longest common subsequences of length 1, "A", "B" and "C".

Example



$X = [a \ b \ a \ c \ a \ c] \quad n=6$

$Y = [c \ a \ d \ a \ c \ c \ a] \quad m=7$

c a d a c c a

	0	0	0	0	0	0	0
a	0						
b	0						
a	0						
c	0						
a	0						
c	0						

- $Lcs[i][j] = \{0, \text{ if } m=0, n=0$
 - $\{1 + Lcs[i-1, j-1], \text{ if } (X[i] == Y[j] \ \& \ i > 0, j > 0)$
 - $\{\text{Max}\{Lcs[i, j-1], Lcs[i-1, j]\}, \text{ if } (X[i] \neq Y[j])$

LCS(X[], Y[], n,m)

{ Lcs[n+1][m+1], path[n+1][m+1]

for(i=0;i<n;i++) Lcs[i][0]=0; for(i=0;i<m;i++) Lcs[0][i]=0;

for(i=1;i<n;i++)

{ for(i=1;i<m;i++)

{ if(X[i-1]==Y[j-1])

Lcs[i][j]=1+Lcs[i-1][j-1]; path[i][j]="\";

Else if(Lcs[i][j-1]>=Lcs[i-1][j])

Lcs[i][j]=Lcs[i][j-1]; path[i][j]="-";

Else

Lcs[i][j]=Lcs[i-1][j]; path[i][j]="|";

}}

}

```
void printLCS(x[], n, m path[][])  
{  if(path[n][m]==“\”)  
    {  printLCS(x, n-1,m-1,path);  
      print(x[n]);  
    }  
  else if(path[n][m]==“-”)  
    printLCS(x, n,m-1,path);  
  else  
    printLCS(x, n-1,m,path);  
}
```



- Set of items $\{x_1, x_2, x_3, \dots, x_n\}$
- Their weight $\{w_1, w_2, w_3, \dots, w_n\}$
- Profit $\{p_1, p_2, p_3, \dots, p_n\}$
- Objective is to select the items in such a way that for a given bag weight, you need to get maximum profit.

Example

- Weights $wt = \{3, 4, 6, 5\}$ $n=4, W=8$
- Profits $p = \{2, 3, 1, 4\}$
- Create table $V[n+1][W+1]$

if($wt[i-1] \leq j$)

$V[i][j] = \max(p[i-1] + V[i-1][j - wt[i-1]], V[i-1][j]);$

else

$V[i][j] = V[i-1][j];$

wt	p
3	2
4	3
6	1
5	4

	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0								
2	0								
3	0								
4	0								

Knapsack(w[], p[], n, W)

Knapsack(wt[], p[], n, W)

```
{ V[n+1][W+1]
  for(i=0; i<=n; i++)
  { for(j=0; j<=W; j++)
    { if(i==0 || j==0)
      V[i][j]=0;
      else if(wt[i-1]<=j)
        V[i][j]= max(p[i-1]+V[i-1][j-wt[i-1]], V[i-1][j]);
      else
        V[i][j]=V[i-1][j];
    }
  }
}
```

	wt	p
0	3	2
1	4	3
2	6	1
3	5	4

	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0								
2	0								
3	0								
4	0								

	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	0	0	2	2	2	2	2	2
2	0	0	0	2	3	3	3	5	5
3	0	0	0	2	3	3	3	5	5
4	0	0	0	2	3	4	4	5	6

```
printKnapsack(V[][])
```

```
{  selected[n]=0; j=W; profit=V[n][W];
```

```
  for(i=n; i>0 &&profit>0; i--)
```

```
  {  if(V[i][j]!=V[i-1][j])
```

```
    {  selected[i-1]=1;
```

```
      profit=profit-p[i-1];  }
```

```
  else
```

```
    selected[i-1]=0;
```

```
    for(k=0; k<W; k++)
```

```
    {  if(V[i-1][k]==profit)
```

```
      j=k; break;
```

```
    }
```

```
}}
```

	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	0	0	2	2	2	2	2	2
2	0	0	0	2	3	3	3	5	5
3	0	0	0	2	3	3	3	5	5
4	0	0	0	2	3	4	4	5	6

Traveling Salesman Problem



Set $\{S\} = \{A, B, C, D, E\}$

$G(i, \{S-i\}) = \min\{w[i, k] + G(k, \{S\} - k)\}$,

where $k \in \{S\}$

& $G()$ is a function

& $W[]$ is weight matrix

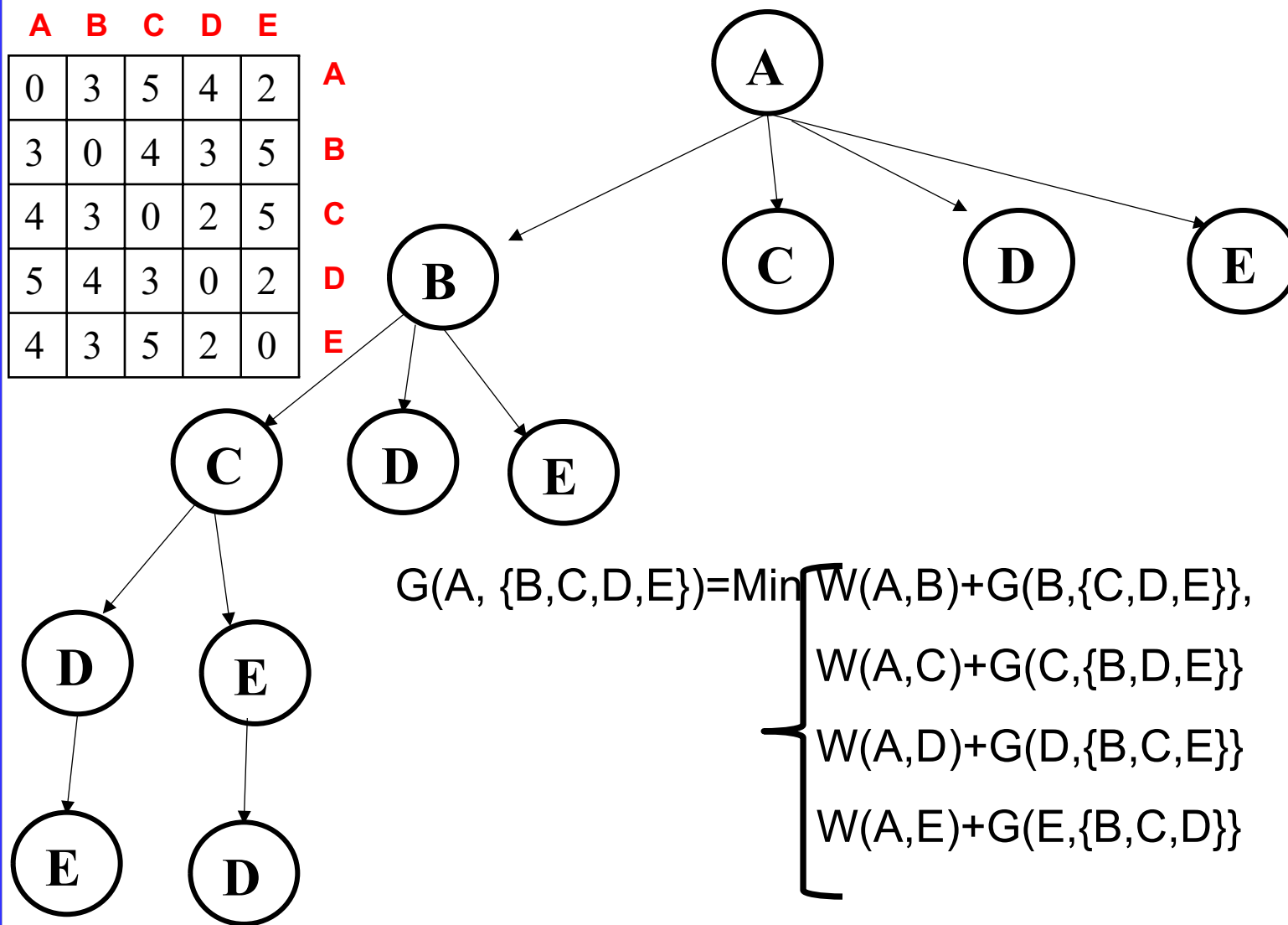
	A	B	C	D	E
A	0	3	5	4	2
B	3	0	4	3	5
C	4	3	0	2	5
D	5	4	3	0	2
E	4	3	5	2	0

$G(A, \{B, C, D, E\}) = \min\{w[A, K] + G((K, \{B, C, D, E\}) - \{K\})$

$K \in \{S\}$

Example

A	B	C	D	E	
0	3	5	4	2	A
3	0	4	3	5	B
4	3	0	2	5	C
5	4	3	0	2	D
4	3	5	2	0	E



Example

A	B	C	D	E
0	3	5	4	2
3	0	4	3	5
4	3	0	2	5
5	4	3	0	2
4	3	5	2	0

A
B
C
D
E
